

Technische Hochschule Würzburg-Schweinfurt
Fakultät Informatik und Wirtschaftsinformatik

Projektdokumentation

Workflowbasierte Rückbestätigung von Lieferterminen

vorgelegt an der Technischen Hochschule Würzburg-Schweinfurt
in der Fakultät Informatik und Wirtschaftsinformatik
im Rahmen der Projektarbeit

Name 1

Name 2

Name 3

Betreuerin: Frau Prof. Dr. Saueressig

Technischer Betreuer: Herr Prof. Dr. Braun

Abgabedatum: 16.09.2026

Inhaltsverzeichnis

1	Einleitung	6
1.1	Motivation und Problemstellung	6
1.2	Zielsetzung	6
1.3	Methodische Vorgehensweise	6
2	IST-Analyse	6
2.1	Bestehender Avisierungsprozess	6
2.2	Bestehende Kommunikationskanäle	6
2.3	Schwachstellen und Optimierungspotenziale	6
2.4	Funktionale Anforderungen	7
2.5	Nichtfunktionale Anforderungen	7
3	SOLL-Konzept	7
3.1	Zielbild des Avisierungsprozesses	7
3.2	SOLL-Prozess	7
3.2.1	Prozessablauf	7
3.2.2	BPMN-Modell	8
4	Systemarchitektur	8
4.1	Systemkontext und Systemgrenzen	9
4.2	Systemkomponenten und Verantwortlichkeiten	10
4.3	Schnittstellen und Zusammenspiel der Komponenten	11
5	Avisierungsservice – Konzeption und Implementierung	12
5.1	Architektur-, Domänen- und Persistenzentwurf	13
5.1.1	Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeits- richtung	13
5.1.2	Domänen- und Zustandsmodell	14
5.1.3	Persistenzmodell und Schemaentwicklung	15
5.2	Technische Umsetzung des Avisierungsprozesses	16
5.2.1	Übernahme von Avisierungsaufträgen, Validierung und Prozessesstart	16
5.2.2	Dauerhafte Workflow-Steuerung mit Camunda	18
5.2.3	Kundenreaktion und Fristablauf	19
5.2.4	Erneute Avisierung und Ablösung veralteter Anfragen	20
5.2.5	Statusrückmeldung an DISPO	21
5.2.6	Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempoten- zenzgrenzen	21
5.3	Kanalunabhängige Kundenkommunikation	23
5.3.1	Kommunikationsabstraktion und Kanalauswahl	23
5.3.2	E-Mail-Versand und mandantenspezifische SMTP-Konfiguration . .	25
5.3.3	SMS und WhatsApp über Twilio: Technologieauswahl und Integration	27
5.4	Mandantenfähigkeit und Sicherheitskonzept	27
5.4.1	Mandantenkontext und Datenisolation	27
5.4.2	Authentifizierung der DISPO-Schnittstelle	29
5.4.3	Zugriffsschutz des Avisierungsdashboards	30
5.4.4	Tokenbasierter Kundenzugriff	32
5.4.5	Schutz sensibler Konfigurationsdaten	33

5.5	Abfragemodell für das Avisierungsdashboard	34
5.5.1	Tourübersicht, dynamische Filterung, Gruppierung und Paginierung	35
5.5.2	Auftragsdetails und Avisierungshistorie	36
5.6	Lieferverfolgung und Geocoding	37
5.6.1	Daten zur Lieferverfolgung und Fahrerposition	37
5.6.2	Zielkoordinaten und Geocoding	37
5.6.3	Bereitstellung für die Bestätigungsseite	37
5.7	Technische Konfiguration und lokale Integrationsumgebung	37
6	Frontend – Konzeption und Implementierung	38
6.1	Einordnung des Frontends in das Gesamtsystem	38
6.2	Frontend-Architektur und technische Grundlagen	38
6.2.1	Technologieauswahl	38
6.2.2	Seiten-, Komponenten- und Routingstruktur	38
6.2.3	API-Anbindung und Zustandsverwaltung	38
6.2.4	Responsive Design und wiederverwendbare UI-Komponenten	38
6.3	Avisierungsdashboard	38
6.3.1	Geschützter Dashboard-Zugang	38
6.3.2	Tourübersicht und Statusdarstellung	38
6.3.3	Suche, Filterung und Paginierung	38
6.4	Auftragsdetailansicht und erneute Avisierung	38
6.4.1	Auftragsdaten und Avisierungshistorie	38
6.4.2	Erneute Avisierung, Validierung und Fehlerrückmeldung	38
6.5	Bestätigungsseite für Kunden	38
6.5.1	Tokenbasierter Zugriff und Lieferinformationen	38
6.5.2	Bestätigung, Ablehnung und Kundenkommentar	38
6.5.3	Abgeschlossene, abgelaufene und fehlerhafte Anfragen	38
6.6	Live-Tracking	38
6.6.1	Verfügbarkeit und Datenfluss	38
6.6.2	Karten- und Positionsdarstellung	38
6.6.3	Aktualisierung und Fehlerbehandlung	38
7	Test und Validierung	38
7.1	Teststrategie und Testumgebung	38
7.1.1	Testziele, Testebenen und Auswahl der Prüfschwerpunkte	39
7.1.2	Testwerkzeuge und reproduzierbare Testumgebung	39
7.2	Validierung des Avisierungsservice	39
7.2.1	Domänen- und Anwendungslogik	39
7.2.2	REST-Verträge, Eingabevalidierung und Fehlerantworten	39
7.2.3	Authentifizierung, Sitzungsverwaltung und Mandantenisolation	39
7.2.4	Persistenz- und Abfragemodell	39
7.2.5	Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien	39
7.2.6	Ausgehende Schnittstellen und Rückkanäle	39
7.3	End-to-End-Tests	39
7.3.1	Einordnung der Szenariogruppen	39
7.3.2	Testumgebung und technische Struktur	39
7.3.3	Versand und Bereitstellung der Anfrage	40
7.3.4	Kundenzugriff und Anzeige der Lieferdaten	40
7.3.5	Kundenreaktion und DISPO-Rückmeldung	41

7.3.6	Dashboard-Zugang	41
7.3.7	Tourenübersicht, Suche und Filterung	41
7.3.8	Auftragsdetails und erneute Avisierung	42
7.3.9	E-Mail- und SMTP-Einstellungen	42
7.3.10	Lieferverfolgung und Tracking-Verfügbarkeit	42
8	Projektmanagement und Projektreflexion	43
8.1	Projektmanagement	43
8.1.1	Projektphasen	43
8.1.2	Projektplan	43
8.1.3	Projektaufwand	44
8.2	Projektreflexion	44
8.2.1	Herausforderungen	44
8.2.2	Lessons Learned	45
9	Projekttrgebnisse	45
9.1	Gesamtergebnis	45
9.2	Grenzen des Projektergebnisses	45
9.3	Fazit und Ausblick	45
10	Nutzung von Künstlicher Intelligenz	45
11	Verwendete Hilfsmittel	45

1 Einleitung

1.1 Motivation und Problemstellung

Ausgangspunkt des Projekts war die Beobachtung, dass die Rückbestätigung von Lieferterminen in weiten Teilen manuell erfolgte und dadurch mit hohem Abstimmungsaufwand verbunden war. Informationen mussten mehrfach weitergegeben, Rückmeldungen manuell ausgewertet und Statusänderungen separat dokumentiert werden.

Ziel der Projektarbeit war es deshalb, einen strukturierten digitalen Prozess zu entwickeln, der sowohl die fachliche Kommunikation mit dem Kunden als auch die technische Weiterverarbeitung im System unterstützt.

1.2 Zielsetzung

1.3 Methodische Vorgehensweise

2 IST-Analyse

2.1 Bestehender Avisierungsprozess

Im bisherigen Ablauf wurden Liefertermine durch die Disposition vorbereitet und anschließend auf direktem Weg an die Kunden kommuniziert. Rückmeldungen erfolgten je nach Situation per E-Mail, telefonisch oder über informelle Zwischenabsprachen. Die eingehenden Informationen mussten anschließend manuell ausgewertet und an weitere Beteiligte weitergegeben werden.

Eine durchgängige digitale Verarbeitung des Prozesses war dadurch nur eingeschränkt möglich. Insbesondere bei mehreren gleichzeitigen Lieferungen stieg der Koordinationsaufwand deutlich an.

2.2 Bestehende Kommunikationskanäle

2.3 Schwachstellen und Optimierungspotenziale

Die wesentlichen Schwachstellen lagen in der fehlenden Standardisierung der Rückmeldungen und in den Medienbrüchen zwischen Kommunikation, Bearbeitung und Dokumentation. Daraus ergaben sich Verzögerungen, erhöhter Abstimmungsbedarf und ein größeres Risiko für Missverständnisse im weiteren Ablauf.

Zusätzlich fehlte eine einheitliche Statussicht, sodass der aktuelle Bearbeitungsstand eines Liefertermins nicht jederzeit direkt nachvollzogen werden konnte.

2.4 Funktionale Anforderungen

ID	Beschreibung
A1	Das System versendet einen Bestätigungslink über E-Mail, SMS oder WhatsApp.
A2	Der Kunde kann über eine Bestätigungsseite auf den vorgeschlagenen Liefertermin reagieren.
A3	Nach Eingang der Kundenrückmeldung wird der Status automatisch im System aktualisiert.
A4	Bei bestätigten Lieferungen kann am Liefertag eine Fahrerkartenansicht bereitgestellt werden.
A5	Die Disposition erhält über ein Dashboard eine Übersicht aller relevanten Vorgänge und Statuswerte.

2.5 Nichtfunktionale Anforderungen

3 SOLL-Konzept

3.1 Zielbild des Avisierungsprozesses

3.2 SOLL-Prozess

3.2.1 Prozessablauf

Im SOLL-Prozess wird die Kommunikation mit dem Kunden standardisiert ausgelöst und systemseitig begleitet. Nach Eingang eines relevanten Vorgangs wird automatisch eine Benachrichtigung mit einem personalisierten Link versendet. Der Versand kann je nach hinterlegtem Kommunikationsweg über E-Mail, SMS oder WhatsApp erfolgen. Über diesen Link gelangt der Kunde auf eine Bestätigungsseite, auf der er strukturiert auf den vorgeschlagenen Liefertermin reagieren kann.

Diese Rückmeldung wird automatisiert verarbeitet, in einen Status überführt und den beteiligten Stellen ohne manuelle Zwischenschritte bereitgestellt. Dadurch entsteht ein durchgängiger Prozess von der Auslösung bis zur Anzeige des aktuellen Bearbeitungsstands. Erfolgt eine Bestätigung, kann am Tag der Lieferung zusätzlich eine Fahrerkartenansicht bereitgestellt werden, um den Lieferkontext für den Kunden transparent darzustellen. Parallel dazu behält die Disposition über ein Dashboard den Überblick über offene, bestätigte oder problematische Vorgänge.

3.2.2 BPMN-Modell



Abbildung 1: BPMN-Modell des SOLL-Prozesses

4 Systemarchitektur

Auf Grundlage des in Kapitel 3 beschriebenen SOLL-Prozesses wird im Folgenden die technische Gesamtarchitektur der digitalen Lieferavisierung dargestellt. Zunächst werden die Systemgrenze und das externe Umfeld abgegrenzt. Darauf aufbauend werden die zentralen Systemkomponenten, ihre Verantwortlichkeiten sowie die relevanten Schnittstellen und Interaktionsabläufe erläutert. Die Kapitel 5 und 6 vertiefen anschließend die interne Konzeption und Implementierung des Avisierungsservice beziehungsweise des Webfrontends.

4.1 Systemkontext und Systemgrenzen

Für die Betrachtung des Systemkontexts wird die digitale Lieferavisierung zunächst als Blackbox aufgefasst. Im Mittelpunkt steht damit nicht ihre interne technische Struktur, sondern die Abgrenzung gegenüber den beteiligten Akteuren und externen Systemen. Dadurch wird deutlich, welche Verantwortlichkeiten innerhalb der entwickelten Lösung liegen und welche außerhalb der Systemgrenze verbleiben.

Von besonderer Bedeutung ist die Abgrenzung gegenüber DISPO. DISPO bleibt das führende System für Auftrags-, Tour- und Lieferplanungsdaten. Die Touren- und Lieferplanung einschließlich der Zuordnung von Fahrzeugen, Liefertagen und Lieferzeitfenstern erfolgt weiterhin dort. Die digitale Lieferavisierung übernimmt die für den Avisierungsprozess erforderlichen Daten und führt den avisierungsspezifischen Prozess- und Rückbestätigungsstatus. Das resultierende Avisierungsergebnis wird anschließend an DISPO zurückgemeldet. Für die Lieferverfolgung stellt DISPO zudem die aktuelle Fahrerposition bereit. Durch diese Aufgabenteilung bleibt die Datenhoheit über die Lieferplanung bei DISPO, während die neue Lösung auf den digitalen Avisierungsprozess und die dafür bereitgestellten Benutzeroberflächen begrenzt wird.

Der Disponent arbeitet primär in DISPO. Dort werden Lieferaufträge geplant und bearbeitet sowie der Avisierungsprozess fachlich ausgelöst. Für die Bearbeitung und Überwachung der avisierungsspezifischen Informationen steht ihm zusätzlich das Avisierungsdashboard zur Verfügung.

Der Kunde greift über einen individuellen Link auf die Bestätigungsseite zu. Eine direkte Interaktion mit DISPO findet nicht statt.

Für die Kundenbenachrichtigung werden externe Kommunikationsdienste angebunden. E-Mails werden über eine externe SMTP-Infrastruktur versendet. Für SMS und WhatsApp wird ein Messaging-Dienst (Twilio) angebunden.

Ein externer Geocoding-Dienst (Nominatim) unterstützt die Lieferverfolgung, indem er aus der Lieferadresse die geografischen Zielkoordinaten ermittelt. Die aktuelle Fahrerposition wird dagegen aus DISPO bezogen.

Abbildung 2 stellt die digitale Lieferavisierung als Blackbox in ihrem Systemkontext dar. Sie zeigt die beteiligten Akteure sowie die Kommunikationsbeziehungen zu DISPO und den angebundenen externen Diensten.

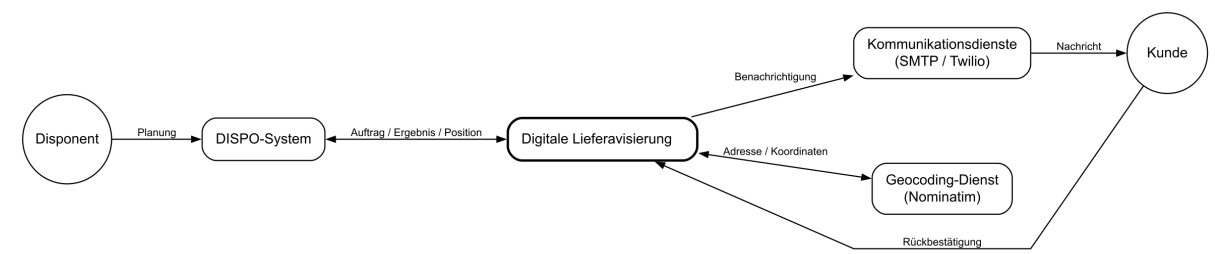


Abbildung 2: Systemkontext der digitalen Lieferavisierung

Damit bildet die digitale Lieferavisierung eine abgegrenzte Integrationslösung zwischen DISPO, den beteiligten Benutzern und den externen technischen Diensten.

4.2 Systemkomponenten und Verantwortlichkeiten

Innerhalb der in Abschnitt 4.1 abgegrenzten digitalen Lieferavisierung übernehmen das Webfrontend, der Avisierungsservice und die PostgreSQL-Datenbank jeweils die Benutzerinteraktion, die fachliche Verarbeitung und die dauerhafte Datenhaltung. Der Avisierungsservice bildet das Backend der digitalen Lieferavisierung, während das Webfrontend die Benutzeroberflächen bereitstellt. Abbildung 3 zeigt ihre Einordnung innerhalb der Systemgrenze.

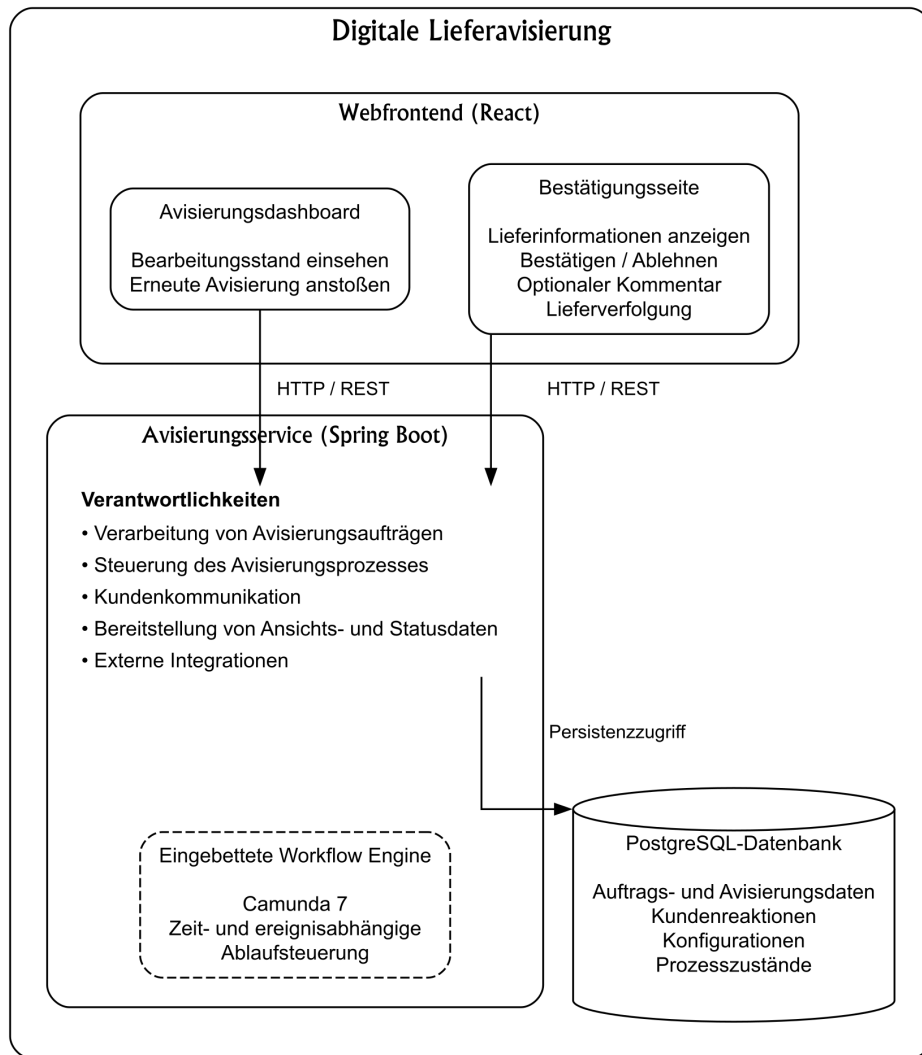


Abbildung 3: Architekturübersicht der digitalen Lieferavisierung

Das **Webfrontend** ist als React-Anwendung umgesetzt und umfasst zwei fachlich getrennte Bereiche: das Avisierungsdashboard für den Disponenten sowie die Bestätigungsseite für den Kunden. Das Avisierungsdashboard ermöglicht die Einsicht in den Bearbeitungsstand und das Anstoßen einer erneuten Avisierung. Über die Bestätigungsseite können Lieferinformationen eingesehen, Lieferzeitfenster bestätigt oder abgelehnt sowie optionale Kommentare übermittelt werden. Für bestätigte Vorgänge steht am Liefertag zusätzlich eine Lieferverfolgung zur Verfügung, sofern die erforderlichen Positionsdaten verfügbar sind.

Das Webfrontend bezieht Daten vom Avisierungsservice und übermittelt Benutzeraktionen an diesen.

Der **Avisierungsservice** bildet die zentrale fachliche und integrative Komponente der Lösung. Als Spring-Boot-Anwendung verarbeitet er die aus DISPO übermittelten Avisierungsaufträge, steuert den Avisierungsprozess, verarbeitet Kundenreaktionen, koordiniert die Kundenkommunikation und stellt die benötigten Ansichts- und Statusdaten bereit. Darüber hinaus bindet er externe technische Dienste an und übermittelt das resultierende Avisierungsergebnis an DISPO. Die interne Architektur des Avisierungsservice wird in Kapitel 5 vertieft.

Die in den Avisierungsservice eingebettete **Workflow Engine Camunda 7** übernimmt die dauerhafte Ablaufkoordination des Avisierungsprozesses. Die konkrete Workflow-Implementierung wird in Abschnitt 5.2 erläutert.

Die **PostgreSQL-Datenbank** dient als zentraler Persistenzspeicher innerhalb der digitalen Lieferavisierung und speichert die für den Avisierungsprozess benötigten Anwendungs- und Prozessdaten.

Durch diese Aufteilung werden Benutzerinteraktion, fachliche Zustandsführung, Ablaufkoordination und persistente Datenhaltung voneinander abgegrenzt.

4.3 Schnittstellen und Zusammenspiel der Komponenten

Im Folgenden werden die wesentlichen Kommunikationsbeziehungen innerhalb der digitalen Lieferavisierung sowie zu den angebundenen externen Systemen betrachtet. Abbildung 4 zeigt zunächst die wesentlichen Schnittstellen und Datenflussrichtungen. Abbildung 5 ergänzt diese statische Sicht anschließend um den zeitlichen Ablauf eines exemplarischen Avisierungsprozesses.

Die Kommunikation mit DISPO umfasst mehrere fachlich unterschiedliche Vorgänge. DISPO übermittelt Avisierungsaufträge an den Avisierungsservice und initialisiert bei Bedarf den Zugang zum Avisierungsdashboard. Die spätere Rückmeldung eines Avisierungsergebnisses erfolgt dagegen in Gegenrichtung. Für die Lieferverfolgung fragt der Avisierungsservice zusätzlich die aktuelle Fahrerposition bei DISPO an und erhält die zugehörigen Positionsdaten zurück.

Auch die beiden Bereiche des Webfrontends kommunizieren direkt mit dem Avisierungsservice. Das Avisierungsdashboard übermittelt Datenabfragen und Benutzeraktionen und erhält die benötigten Übersichts- und Detailinformationen zurück. Der Dashboardzugang wird aus DISPO heraus initiiert; nach dem Übergang erfolgt die weitere Kommunikation unmittelbar zwischen Webfrontend und Avisierungsservice. Die Bestätigungsseite wird über den individuellen Link aus der Kundenbenachrichtigung aufgerufen. Sie bezieht Liefer- und Avisierungsdaten sowie Daten zur Lieferverfolgung vom Avisierungsservice und übermittelt die Kundenreaktion über die dafür bereitgestellte Schnittstelle.

Für die Lieferverfolgung werden Daten aus unterschiedlichen Quellen zusammengeführt. Die Fahrerposition stammt aus DISPO, während die geografischen Koordinaten des Lieferziels über den Geocoding-Dienst ermittelt werden. Hierzu übermittelt der Avisierungsservice die Lieferadresse als Geocoding-Anfrage und erhält die bestimmten Zielkoordinaten zurück. Weitere ausgehende Schnittstellen dienen der Kundenkommunikation: E-Mail-Nachrichten

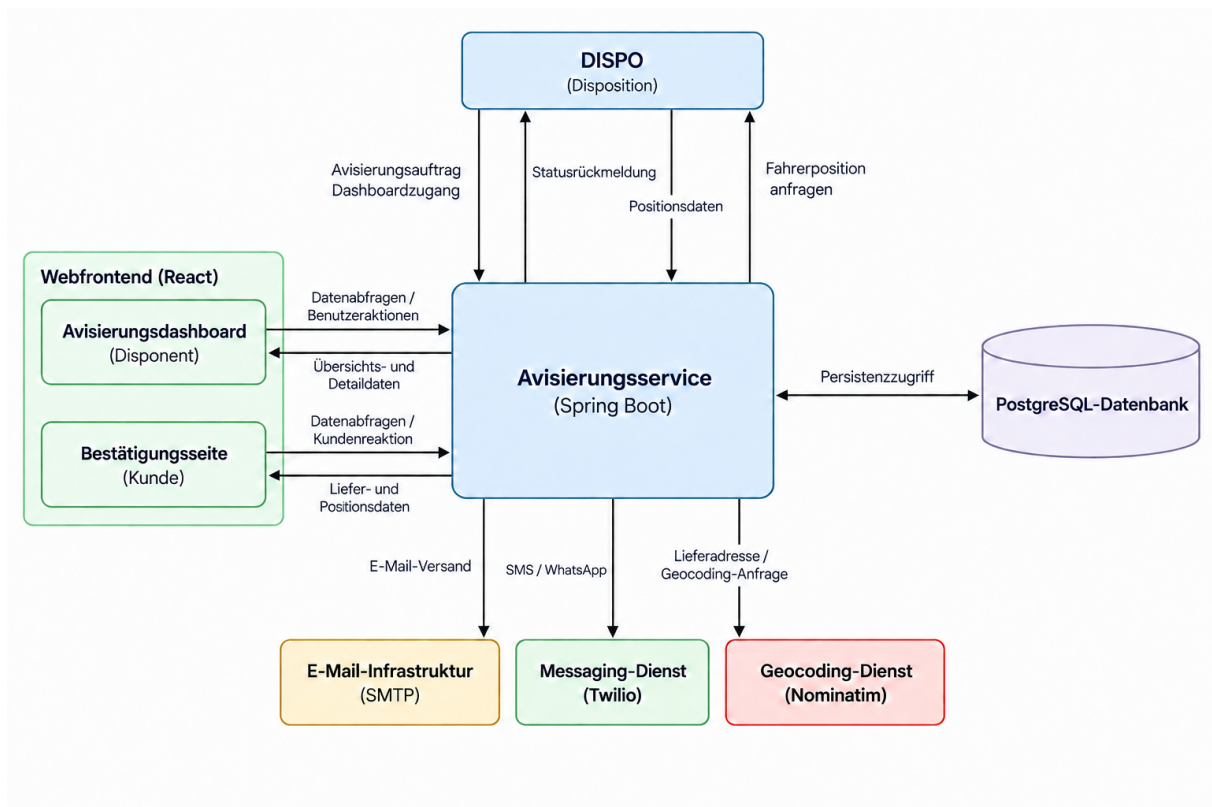


Abbildung 4: Wesentliche Schnittstellen der digitalen Lieferavisierung

werden über die angebundene SMTP-Infrastruktur versendet, SMS- und WhatsApp-Nachrichten über den Messaging-Dienst. Der Zugriff auf die PostgreSQL-Datenbank erfolgt ausschließlich über den Avisierungsservice und dient der persistenten Speicherung der für den Prozess benötigten fachlichen und technischen Zustände.

Der Avisierungsprozess besteht nicht aus einem einzelnen synchronen Kommunikationsvorgang, sondern aus mehreren zeitlich getrennten Interaktionen. Insbesondere die Übermittlung eines Avisierungsauftrags und die spätere Rückmeldung des Ergebnisses an DISPO bilden getrennte Vorgänge. Ebenso erfolgen der Abruf der Bestätigungsseite und die anschließende Übermittlung einer Kundenreaktion über separate Schnittstellen. Abbildung 5 zeigt, wie diese zeitlich getrennten Interaktionen innerhalb eines Avisierungsvorgangs zusammenwirken.

Der Ablauf verdeutlicht die zeitliche Entkopplung zwischen der ursprünglichen Auftragsübernahme und der späteren Rückmeldung des Avisierungsergebnisses. Die dazwischenliegenden Interaktionen werden über einen persistent gespeicherten Prozesszustand miteinander verknüpft. Die interne Umsetzung dieser Koordination wird in Abschnitt 5.2 vertieft.

5 Avisierungsservice – Konzeption und Implementierung

Dieses Kapitel beschreibt die Konzeption und technische Umsetzung des Avisierungsservice. Aufbauend auf der zuvor dargestellten Gesamtarchitektur werden zunächst die architekto-

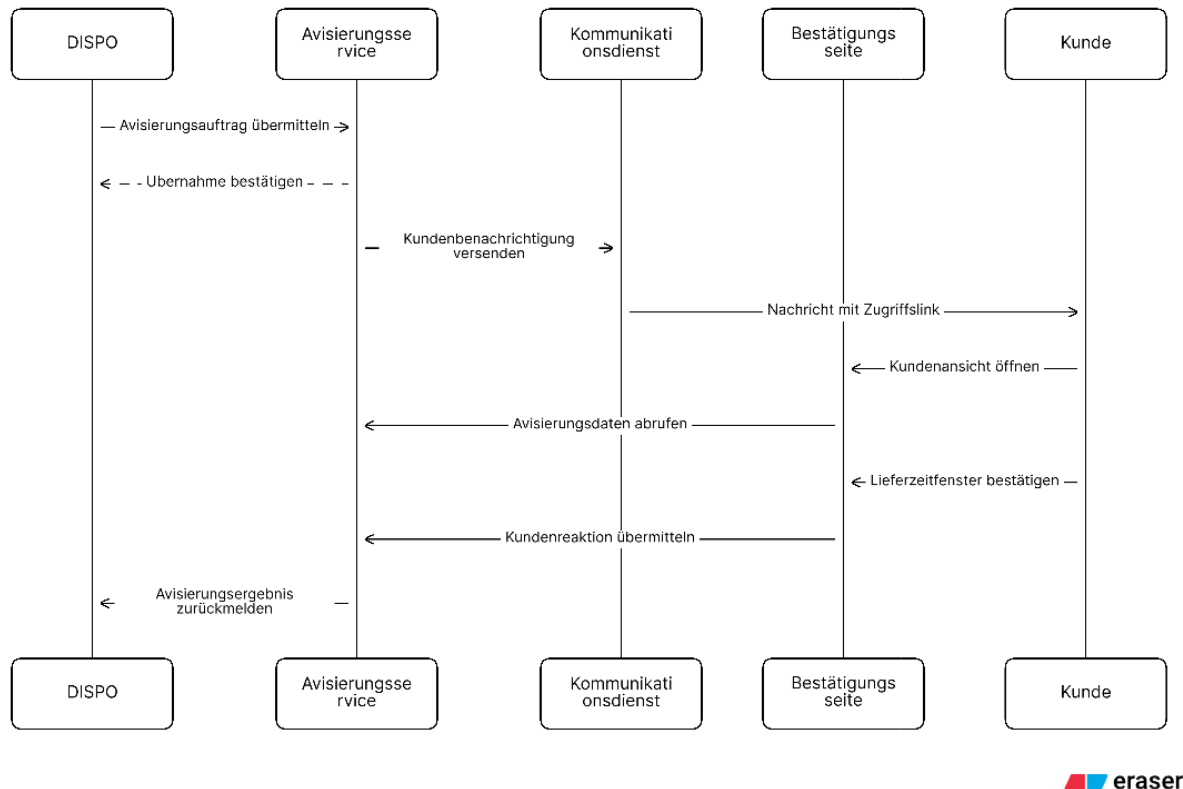


Abbildung 5: Technischer Interaktionsablauf eines Avisierungsprozesses

nischen, fachlichen und persistenzbezogenen Grundlagen erläutert. Anschließend werden der Avisierungsprozess, die Kundenkommunikation, die Mandanten- und Sicherheitsmechanismen, das Abfragemodell, die Lieferverfolgung und das Geocoding sowie die technische Konfiguration und lokale Integrationsumgebung betrachtet. Im Mittelpunkt stehen die Entwurfsentscheidungen, ihre Umsetzung und ihre Auswirkungen auf Verantwortungsgrenzen, Datenkonsistenz und Betrieb.

5.1 Architektur-, Domänen- und Persistenzentwurf

5.1.1 Architekturprinzipien, Verantwortungsgrenzen und Abhängigkeitsrichtung

Der Avisierungsservice ist als Java-Anwendung mit Spring Boot umgesetzt, das als technischer Rahmen des Projekts vorgegeben war. Die Architektur orientiert sich am Ports-and-Adapters-Ansatz (**cockburn2005hexagonal**).

Die Quellcodeorganisation unterscheidet die Bereiche **domain**, **application**, **adapter** und **configuration**. Der Domain Layer enthält die fachlichen Objekte, Zustände und unmittelbar damit verbundenen Regeln, insbesondere **Order**, **ConfirmationRequest** und **CustomerResponse**. Der Application Layer koordiniert darauf aufbauend die Anwendungsfälle.

Eingehende Aufrufe werden durch Inbound-Adapter wie REST-Controller oder Camunda-Delegates entgegengenommen. Ausgewählte ausgehende Integrationen werden über Outbound-

Ports abstrahiert, darunter `NotificationService` für die Kundenkommunikation und `DispoStatusCallbackService` für die Statusrückmeldung an DISPO. Der Bereich `configuration` enthält die Spring-spezifische Konfiguration und Verdrahtung.

Für die portbasiert angebundenen Teile verläuft die statische Abhängigkeitsrichtung von den Inbound-Adaptern über den Application Layer zum Domain Layer beziehungsweise vom Application Layer über einen Outbound-Port zum implementierenden Adapter.

Für den Persistenzzugriff wurde diese Trennung bewusst nicht angewendet. Der festgelegte Persistenzstack basiert auf Jakarta Persistence (JPA) und Spring Data JPA. Application-Services greifen daher teilweise direkt auf Spring-Data-JPA-Repositories und Abstraktionen wie `Pageable` zu; zugleich definieren die Domänenklassen ihre relationale Abbildung über Jakarta-Persistence-Annotationen.

Abbildung ?? zeigt die resultierende Adapter- und Abhängigkeitsstruktur des Avisierungsservice.

Auf separate Repository-Ports, Persistence-Entities und die damit verbundene Mappinglogik wurde verzichtet. Eine Austauschbarkeit der Persistenztechnologie war im Projektumfang nicht gefordert und hätte zusätzlichen Struktur- und Mappingaufwand verursacht.

Diese Entscheidung koppelt den Application Layer stärker an Spring Data und das Domänenmodell an Jakarta Persistence. Änderungen des Persistenzstacks oder seiner Mappinganforderungen können daher Anpassungen in diesen Bereichen erfordern.

Die Umsetzung entspricht somit keiner strikt technologieunabhängigen hexagonalen Architektur, sondern kombiniert die portbasierte Entkopplung ausgewählter Integrationen mit der direkten Einbindung des festgelegten Persistenzstacks.

5.1.2 Domänen- und Zustandsmodell

Das Domänenmodell trennt den aus DISPO übernommenen Auftrag, einzelne Rückbestätigungsanfragen und die darauf bezogenen Kundenreaktionen. Fachliche, technische und prozessbezogene Zustände werden jeweils dem Objekt zugeordnet, dessen Verantwortung sie beschreiben.

`Company` bildet den Mandantenbezug. Jeder `Order` ist genau einer `Company` zugeordnet; auch die externe Auftragsidentität wird in diesem Mandantenkontext betrachtet. Die technische Mandantentrennung wird in Abschnitt 5.4.1 behandelt.

`Order` repräsentiert den aus DISPO übernommenen fachlichen Auftrag mit den für die Avisierung relevanten Auftrags- und Tourdaten. Sein `confirmationStatus` beschreibt den auftragsbezogenen Stand des Avisierungsprozesses.

Eine konkrete Avisierung wird durch `ConfirmationRequest` abgebildet. Sie enthält unter anderem Token, Kommunikationskanal, Lieferfenster und Antwortfrist. Ein `Order` kann mehrere aufeinanderfolgende `ConfirmationRequest`-Objekte besitzen, sodass frühere Anfragen als Avisierungshistorie erhalten bleiben.

Der `deliveryStatus` eines `ConfirmationRequest` beschreibt den technischen Versandstand. `SENT` bedeutet lediglich, dass der Versandaufwurf ohne gemeldeten technischen Fehler abgeschlossen wurde; eine tatsächliche Zustellung oder Kenntnissnahme des Kunden wird dadurch nicht nachgewiesen.

Das zusätzliche Merkmal `active` kennzeichnet unabhängig vom Versandstatus, ob die Anfrage noch als gültige Grundlage für eine Kundenreaktion dient. Die konkreten Zustandsübergänge werden in Abschnitt 5.2 beschrieben.

Eine `CustomerResponse` ist genau derjenigen `ConfirmationRequest` zugeordnet, auf die sich die Kundenentscheidung bezieht. Dadurch bleibt nachvollziehbar, auf welches Lieferfenster der Kunde reagiert hat. Pro Anfrage kann höchstens eine Kundenreaktion gespeichert werden; zusätzlich können ein Kommentar und der Eingangszeitpunkt `receivedAt` erfasst werden.

Die eingebetteten Fachstrukturen `Tour` und `DeliverySlot` besitzen keinen eigenständigen Lebenszyklus. `Tour` umfasst Tournummer und Fahrzeugkennzeichen. `DeliverySlot` repräsentiert Datum, Beginn und Ende des vorgeschlagenen Lieferfensters und stellt sicher, dass dessen Beginn vor dem Ende liegt.

`CompanyEmailSettings` enthält die mandantenspezifische E-Mail-Konfiguration und ist höchstens einmal einer `Company` zugeordnet. Die technische Ausgestaltung wird in Abschnitt 5.3.2, der Schutz sensibler Werte in Abschnitt 5.4.5 behandelt.

Tabelle ?? fasst die getrennt modellierten Zustandsaspekte zusammen.

Die Trennung verhindert eine Vermischung fachlicher und technischer Zustände. Ein fehlgeschlagener Versand stellt beispielsweise keine Ablehnung des Kunden dar; ebenso bedeutet ein technisch erfolgreicher Versand noch keine Bestätigung des vorgeschlagenen Liefertermins.

5.1.3 Persistenzmodell und Schemaentwicklung

Das Persistenzmodell basiert auf PostgreSQL. Die relationale Abbildung erfolgt über Jakarta Persistence mit Hibernate als Persistence Provider; für den Datenzugriff wird Spring Data JPA eingesetzt. Wie in Abschnitt 5.1.1 beschrieben, befinden sich die JPA-Mappings unmittelbar in den Domänenklassen.

`Company`, `Order`, `ConfirmationRequest` und `CustomerResponse` werden in den Tabellen `company`, `order_snapshot`, `confirmation_request` und `customer_response` persistiert. Die mandantenspezifische E-Mail-Konfiguration liegt in `company_email_settings`.

Die eingebetteten Strukturen `Tour` und `DeliverySlot` besitzen keine eigenen Tabellen. Tournummer und Fahrzeugkennzeichen werden in `order_snapshot` gespeichert; Datum sowie Beginn und Ende des Lieferfensters in `confirmation_request`.

Abbildung ?? zeigt die zentralen anwendungseigenen Tabellen, ihre Beziehungen sowie ausgewählte Schlüssel- und Eindeutigkeitsbedingungen. Von Camunda und Flyway verwaltete technische Tabellen werden nicht dargestellt.

Die Zuordnung eines `Order` zu seiner `Company` erfolgt über `company_id`. Da die aus DISPO übernommene `external_order_id` nur innerhalb eines Mandanten eindeutig sein muss, ist die Kombination aus `company_id` und `external_order_id` eindeutig definiert.

Ein `ConfirmationRequest` verweist über `order_snapshot_id` auf den zugehörigen Auftrag. Sein Token ist schemaweit eindeutig; ob er für einen Kundenzugriff noch gültig ist, entscheidet davon unabhängig die Anwendungslogik. Die Eindeutigkeit von `customer_response.confirmationRequestToken` begrenzt jede Anfrage auf höchstens eine Kundenreaktion. Analog stellt die Eindeutigkeit

von `company_email_settings.company_id` höchstens einen E-Mail-Konfigurationsdatensatz pro Mandant sicher.

Zusätzliche CHECK-Constraints sichern ausgewählte strukturelle Gültigkeitsregeln ab. So muss `response_deadline_hours` positiv sein; für die E-Mail-Konfiguration bestehen unter anderem Bedingungen für SMTP-Port und erforderliche Zugangsdaten bei aktivierter Authentifizierung.

Nicht alle fachlichen Invarianten werden zusätzlich in der Datenbank abgebildet. Die Reihenfolge von Beginn und Ende eines `DeliverySlot` wird beispielsweise im Domänenmodell geprüft. Die Datenbank sichert damit insbesondere referenzielle Integrität, Eindeutigkeit und ausgewählte strukturelle Wertebereiche.

Ergänzende Indizes unterstützen insbesondere mandanten- und tourbezogene Abfragen sowie den Zugriff auf die Rückbestätigungsanfragen eines Auftrags.

Die Schemaentwicklung wird mit Flyway gesteuert. Änderungen werden als versionierte Migrationen in definierter Reihenfolge ausgeführt und in der Flyway-Schemahistorie dokumentiert, wodurch sie reproduzierbar auf unterschiedliche Laufzeitumgebungen übertragen werden können (**redgate2024versionedMigrations**). Hibernate ist mit `ddl-auto: validate` konfiguriert und verändert das Schema nicht selbst, sondern prüft beim Anwendungsstart dessen Kompatibilität mit den JPA-Mappings.

Ausgewählte Migrationen zeigen die Weiterentwicklung des Persistenzmodells. Mit V6 wurde die Mandantenfähigkeit eingeführt: Die Tabelle `company` und `company_id` in `order_snapshot` wurden ergänzt und die zuvor globale Eindeutigkeit der `external_order_id` durch die mandantenbezogene Eindeutigkeit von `company_id` und `external_order_id` ersetzt.

Mit V9 wurde `company_email_settings` für die mandantenspezifische E-Mail-Konfiguration ergänzt. V10 führte den technischen `delivery_status` in `confirmation_request` ein und bildete damit die technische Versandverarbeitung als eigenen Zustand auch im Persistenzmodell ab.

Das Persistenzmodell überträgt damit die zentralen Beziehungen des Domänenmodells und ausgewählte Integritätsregeln in die relationale Datenstruktur; seine Weiterentwicklung erfolgt kontrolliert über Flyway.

5.2 Technische Umsetzung des Avisierungsprozesses

Der Avisierungsprozess umfasst die Übernahme des Avisierungsauftrags aus DISPO, die Benachrichtigung des Kunden, die Verarbeitung der Rückbestätigung oder eines Fristablaufs sowie die abschließende Statusrückmeldung an DISPO. Er verbindet damit die synchrone Auftragsübernahme mit einer nachgelagerten, zeitlich verteilten Verarbeitung. Die folgenden Abschnitte beschreiben diesen technischen Lebenszyklus bis zur Behandlung von Fehlern, Wiederholungen und Nebenläufigkeit.

5.2.1 Übernahme von Avisierungsaufträgen, Validierung und Prozessstart

Ein Avisierungsauftrag wird über eine REST-Schnittstelle aus DISPO übernommen und zusammen mit dem bereits bestimmten Mandantenkontext an die Anwendungslogik übergeben. Ob dabei eine neue Rückbestätigungsanfrage und Prozessinstanz entstehen,

hängt vom vorhandenen Verarbeitungszustand ab. Die vorgelagerte Authentifizierung und Ermittlung des Mandantenkontexts werden in Abschnitt 5.4.2 beschrieben.

Der Auftrag enthält die für die Avisierung erforderlichen Auftrags-, Tour-, Kunden-, Kontakt- und Lieferdaten sowie Prozessparameter wie Kommunikationskanal und Antwortfrist.

Die Eingangsdaten werden zunächst an der Schnittstellengrenze strukturell validiert. Dazu gehören Pflichtfelder, Formatvorgaben und Wertebereiche. Fachlich kontextabhängige Regeln werden anschließend in der Anwendungslogik beziehungsweise im Domänenmodell geprüft. So muss für den gewählten Kommunikationskanal die passende Kontaktinformation vorliegen, und das übermittelte Lieferfenster darf zum Zeitpunkt der Auftragsübernahme noch nicht begonnen haben. Dadurch werden strukturelle und fachliche Validierung getrennt, bevor eine neue Prozessverarbeitung initialisiert wird.

Der Mandantenkontext bestimmt den zu verwendenden Datenbestand. Die zugehörige **Company** wird daraus ermittelt; ein bestehender **Order** wird über Mandanten-ID und externe Auftrags-ID gesucht. Eine erneut übertragene externe Auftrags-ID führt daher nicht automatisch zu einem weiteren **Order**-Objekt.

Die weitere Verarbeitung ist zustandsabhängig. Der Avisierungsservice berücksichtigt den bestehenden **Order** und dessen letzte Rückbestätigungsanfrage und vergleicht deren relevante Daten mit dem neu eingegangenen Auftrag. Daraus ergibt sich, ob die bestehende Verarbeitung weiterverwendet oder aktualisiert werden kann oder eine neue Rückbestätigungsanfrage mit neuer Prozessinstanz erforderlich ist. Tabelle ?? fasst diese Entscheidung zusammen.

Eine identische erneute Übertragung erzeugt bei einer bereits laufenden **PENDING**-Anfrage keine zusätzliche Prozessinstanz. Ändern sich die Daten in diesem Zustand, werden Auftrag und bestehende Anfrage aktualisiert, wodurch ihre Zuordnung zur vorhandenen Prozessinstanz erhalten bleibt. Auch eine aktive **SENT**-Anfrage oder ein weiterhin verwendbares abgeschlossenes Ergebnis kann bei unveränderten Daten wiederverwendet werden.

Ist eine neue Rückbestätigungsanfrage erforderlich, wird ein Token für den späteren Kundenzugriff erzeugt und ein neuer **ConfirmationRequest** mit Kommunikationskanal, Lieferfenster und Antwortfrist im Versandstatus **PENDING** gespeichert. Zu diesem Zeitpunkt ist die Anfrage noch nicht für Kundenreaktionen aktiviert. Die nach der Persistierung verfügbare interne ID dient zur Zuordnung und zum Start der Prozessinstanz; der davon unabhängige Token wird ausschließlich für den Kundenzugriff verwendet (vgl. Abschnitt 5.4.4).

Der Prozessstart erfolgt noch innerhalb der Auftragsübernahme, der eigentliche Nachrichtenversand jedoch erst nach einer asynchronen Fortsetzungsstelle. Der HTTP-Aufruf bestätigt somit die Übernahme des Auftrags, während die von externen Kommunikationssystemen abhängige Verarbeitung nachgelagert durch die Workflow Engine ausgeführt wird.

Dies spiegelt sich in der HTTP-Semantik wider. Befindet sich der Auftrag nach der Verarbeitung weiterhin im Status **OPEN**, antwortet die DISPO-Schnittstelle mit **202 Accepted**. Dies bedeutet weder, dass die Nachricht bereits versendet wurde, noch zwingend, dass eine neue Prozessinstanz entstanden ist. Ein bereits vorhandener und weiterhin verwendbarer Zustand führt dagegen zu **200 OK**; dies umfasst sowohl eine aktive **SENT**-Anfrage als auch die Ergebnisse **CONFIRMED** oder **REJECTED**.

Die Auftragsübernahme trennt damit strukturelle und fachliche Validierung, ordnet den Auftrag einem Mandanten zu und entscheidet zustandsabhängig über Wiederverwendung, Aktualisierung oder Neuerzeugung der weiteren Verarbeitung.

Fehler an den REST-Schnittstellen werden zentral in ein einheitliches Fehlerformat überführt. Strukturelle Validierungsfehler, fachliche Konflikte und technische Fehler werden dabei unterschiedlichen HTTP-Statuscodes und Fehlerkennungen zugeordnet. Dadurch bleibt die Fehlerbehandlung der Controller von den einzelnen Ausnahmerearten getrennt, und die Schnittstellen liefern ein konsistentes Fehlerformat.

5.2.2 Dauerhafte Workflow-Steuerung mit Camunda

Der Avisierungsprozess umfasst Wartezeiten und Ereignisse, die unabhängig vom ursprünglichen HTTP-Aufruf eintreten. Nach dem Versand wartet er beispielsweise auf eine Kundenreaktion, den Ablauf der Antwortfrist oder die Ablösung durch eine neuere Avisierung. Diese Anforderungen erfordern dauerhaft persistierbare Wartezustände sowie zeit- und ereignisabhängige Fortsetzungen.

Dafür wurde eine workflowbasierte Prozesssteuerung gewählt. Eine rein anwendungsseitige Lösung hätte Wartezustände, Timer und Ereigniskorrelation zusätzlich in Anwendungslogik und Persistenz abbilden müssen. Eine Workflow Engine ermöglicht dagegen ein ausführbares Prozessmodell und übernimmt dessen Ablaufkoordination. Dem stehen eine zusätzliche Laufzeitkomponente, höherer Integrations- und Betriebsaufwand sowie eine teilweise Bindung an enginespezifische Schnittstellen gegenüber.

Als konkrete Workflow Engine wird Camunda 7 eingesetzt. Sie übernimmt die Ausführung der BPMN-Prozessdefinition, persistente Wartezustände, Timer, Nachrichtenkorrelation und die Anbindung von Service Tasks an Java-Code. Die Workflow Engine ist in die Spring-Boot-Anwendung integriert; die BPMN-Modelle werden beim Start automatisch deployt. Die Runtime API wird unter anderem für Prozessstarts und Nachrichtenkorrelation verwendet.

Während Kapitel 3 den fachlichen SOLL-Prozess beschreibt, zeigt Abbildung ?? dessen technische Konkretisierung durch Service Tasks, Wartezustände, Nachrichtenereignisse, Timer und Fehlerpfade.

Das BPMN-Modell wird unter dem Schlüssel `confirmation-request-process` deployt. Jede Prozessinstanz repräsentiert einen konkreten Avisierungsvorgang und ist über die interne `ConfirmationRequest.id` als Business Key einer Rückbestätigungsanfrage zugeordnet. Derselbe Schlüssel wird zusammen mit dem jeweiligen Message-Namen zur Korrelation von Kundenreaktionen oder Ablösungsereignissen verwendet.

Der fachliche Zustand verbleibt in den Domänenobjekten und der Anwendungsdatenbank. Die Workflow Engine verwaltet dagegen den Ausführungszustand der Prozessinstanz und die für die Ablaufsteuerung erforderlichen Prozessvariablen. Sie ergänzt somit das fachliche Datenmodell um den für die Orchestrierung notwendigen Prozesszustand.

Die Integration erfolgt in beide Richtungen. Tabelle ?? fasst die technischen Bindeglieder und Verantwortlichkeiten zusammen.

Java Delegates bilden die Camunda-spezifische Schnittstelle zur Anwendungslogik. Sie lesen Business Key und Prozessvariablen, rufen die zugehörigen Use Cases auf und überführen

deren Ergebnisse in den BPMN-Ablauf. Beim Versand kann das Ergebnis beispielsweise über Prozessvariablen weitergegeben oder als `BpmnError` an ein Boundary Error Event signalisiert werden. Die fachliche Verarbeitung verbleibt damit im Application Layer.

Die Trennung vom ursprünglichen HTTP-Aufruf wird durch eine asynchrone Fortsetzungsstelle vor dem Nachrichtenversand umgesetzt. Der Prozessstart erfolgt noch synchron, die weitere Verarbeitung anschließend als Job der Workflow Engine.

Nach erfolgreichem Versand erreicht die Prozessinstanz ein ereignisbasiertes Gateway und wartet dort auf Kundenreaktion, Fristablauf oder Ablösung. Dabei bleibt kein Anwendungsthread blockiert; die Instanz wird erst durch das eintretende Ereignis fortgesetzt.

Für fehlgeschlagenen Nachrichtenversand enthält das BPMN-Modell einen eigenen Fehlerpfad. Die konkrete Fehlerklassifikation und Wiederholungsstrategie werden in Abschnitt 5.2.6 beschrieben.

Damit verbleibt die fachliche Zustandsführung im Avisierungsservice, während die Workflow Engine die zeit- und ereignisabhängige Ablaufkoordination übernimmt.

5.2.3 Kundenreaktion und Fristablauf

Bevor eine Kundenreaktion übermittelt wird, stellt der Avisierungsservice die für die Bestätigungsseite benötigten Auftrags- und Lieferinformationen anhand des im Bestätigungslink enthaltenen Tokens bereit. Dabei wird ausschließlich die jeweils neueste Rückbestätigungsanfrage eines Auftrags berücksichtigt. Neben den Lieferdaten wird aus dem Zustand der Anfrage und einer gegebenenfalls bereits vorhandenen Kundenreaktion der darzustellende Bestätigungsstatus abgeleitet.

Nach einem technisch erfolgreichen Versand wird die Rückbestätigungsanfrage in den Versandstatus `SENT` überführt, `sentAt` gespeichert und mit `active = true` für Kundenreaktionen aktiviert. Gleichzeitig wechselt der zugehörige `Order` von `OPEN` nach `SENT`.

Mit dem Versand wird der tatsächliche Ablaufzeitpunkt bestimmt. Er entspricht dem früheren Zeitpunkt aus konfigurierter Antwortdauer ab `sentAt` und Beginn des Lieferfensters:

$$\text{expiresAt} = \min\left(\text{sentAt} + \text{konfigurierte Antwortdauer}, \text{Beginn des Lieferfensters}\right)$$

Die Antwortdauer wird über `responseDeadlineHours` angegeben. Dadurch beginnt die Frist erst nach erfolgreichem Versand und endet spätestens mit dem Beginn des Lieferfensters.

Kundenreaktionen werden über den in der Nachricht enthaltenen Token entgegengenommen. Dieser muss zur jeweils neuesten `ConfirmationRequest` des Auftrags gehören, sodass überholte Anfragen nicht mehr beantwortet werden können. Die sicherheitsbezogene Ausgestaltung wird in Abschnitt 5.4.4 beschrieben.

Der Kunde kann den Liefertermin mit `CONFIRM` bestätigen oder mit `REJECT` ablehnen und optional einen Kommentar übermitteln. Vor der Verarbeitung wird geprüft, ob die Anfrage weiterhin aktiv, noch nicht abgelaufen und unbeantwortet ist. Eine Antwort ist nur zulässig, solange `receivedAt < expiresAt` gilt.

Bei einer gültigen Antwort wird der `Order` auf `CONFIRMED` beziehungsweise `REJECTED` gesetzt, die `CustomerResponse` gespeichert und die Anfrage deaktiviert. Ihr technischer Versandstatus bleibt `SENT`, da die Kundenentscheidung den bereits erfolgten Versand nicht verändert.

Anschließend wird die Kundenreaktion mit der Prozessinstanz korreliert. Eine zusätzliche Eingangsbestätigung an den Kunden wird als nachgelagerter Kommunikationseffekt ausgelöst. Schlägt dieser Versand fehl, wird der Fehler protokolliert; die bereits gespeicherte Kundenentscheidung bleibt bestehen.

Der alternative Pfad ist der Fristablauf über das im Workflow hinterlegte Timer-Ereignis. Vor einer fachlichen Zustandsänderung prüft die Anwendungslogik erneut, ob die Anfrage noch aktiv und `expiresAt` tatsächlich erreicht ist. Nur dann wird sie deaktiviert und der `Order` auf `NO_RESPONSE` gesetzt. Der `deliveryStatus` bleibt auch hier `SENT`.

Das Ausbleiben einer Antwort ist damit kein Versandfehler, sondern das fachliche Ergebnis einer zuvor erfolgreich eröffneten Antwortphase. Der Timer bildet lediglich den technischen Auslöser; die Zulässigkeit des Zustandsübergangs entscheidet weiterhin die Anwendungslogik.

Tabelle ?? fasst die resultierenden Zustandsübergänge zusammen.

Eine gültige Kundenentscheidung oder der Fristablauf beendet die aktive Antwortphase. Der Auftrag befindet sich anschließend in `CONFIRMED`, `REJECTED` oder `NO_RESPONSE`; die Anfrage ist deaktiviert und behält ihren technischen Versandstatus `SENT`. Daneben kann eine laufende Anfrage durch eine neuere Avisierung abgelöst werden.

5.2.4 Erneute Avisierung und Ablösung veralteter Anfragen

Eine erneute Verarbeitung kann auf zwei Wegen ausgelöst werden. Einerseits kann DISPO einen bereits bekannten Auftrag erneut übermitteln, beispielsweise aufgrund geänderter Auftrags- oder Avisierungsdaten oder nach einem zuvor erfolglosen Avisierungsversuch. Andererseits kann nach einem endgültigen Zustellfehler oder einem Fristablauf ohne Kundenreaktion eine erneute Avisierung manuell über das Avisierungsdashboard angestoßen werden. Beide Wege können zur Erzeugung einer neuen Rückbestätigungsanfrage führen, unterscheiden sich jedoch hinsichtlich ihrer Änderungsmöglichkeiten und fachlichen Voraussetzungen.

Sind bei der erneuten Übermittlung durch DISPO die Daten einer bereits versendeten Anfrage geändert, kann die bisherige Anfrage nicht weiterverwendet werden. Eine noch aktive Anfrage wird daher deaktiviert und die zugehörige wartende Prozessinstanz über das Ereignis `ConfirmationRequestSuperseded` beendet. Die abgelöste Prozessinstanz endet auf diesem Pfad ohne fachliche Statusrückmeldung an DISPO. Maßgeblich für die weitere Verarbeitung ist anschließend die neu erzeugte Rückbestätigungsanfrage.

Wird aufgrund des bestehenden Zustands eine neue Rückbestätigungsanfrage erzeugt, werden deren anfragespezifische Daten separat gespeichert. Dadurch bleiben das jeweilige Lieferfenster, der Kommunikationskanal, die Antwortfrist und der Versandstatus der einzelnen Versuche nachvollziehbar.

5.2.5 Statusrückmeldung an DISPO

Beim Abschluss des ursprünglichen HTTP-Aufrufs aus DISPO liegt noch kein Rückbestätigungsergebnis vor. Eine Kundenentscheidung kann unmittelbar nach dem Versand, aber auch erst nach längerer Wartezeit eintreten; alternativ kann die Antwortfrist ohne Kundenreaktion ablaufen. Das Integrationskonzept sieht daher einen mandantenspezifisch konfigurierten Callback-Endpunkt vor, über den der Avisierungsservice den später feststehenden Status an DISPO zurückmeldet.

Die Statusrückmeldung ist Bestandteil des regulären Prozesspfads und erfolgt vor dessen Abschluss. Das ausführbare Prozessmodell führt die Zustände **CONFIRMED**, **REJECTED** und **NO_RESPONSE** zu diesem Schritt. Ein technischer Zustellfehler mit dem Versandstatus **FAILED** sowie die Ablösung einer Rückbestätigungsanfrage durch eine neuere Avisierung enden dagegen auf separaten Prozesspfaden. Der Callback-Schritt übergibt den bereits bestimmten Status zusammen mit der Identität der zugrunde liegenden Rückbestätigungsanfrage an die Anwendungslogik zur externen Rückmeldung.

Vor dem HTTP-Aufruf wird geprüft, ob die zugrunde liegende Rückbestätigungsanfrage noch aktuell ist. Hierzu wird die jüngste **ConfirmationRequest** des betreffenden **Order** bestimmt und ihre ID mit der im Business Key der Prozessinstanz enthaltenen Anfrage-ID verglichen. Nur bei Übereinstimmung wird die Rückmeldung ausgeführt.

Diese Prüfung verhindert insbesondere die Übertragung eines bereits überholten Status. Wird beispielsweise nach einer Rückbestätigungsanfrage A aufgrund geänderter Avisierungsdaten eine neue Anfrage B erzeugt und erreicht die Prozessinstanz von A erst danach den Callback-Schritt, wird die Rückmeldung von A unterdrückt. Anfrage A ist zu diesem Zeitpunkt nicht mehr die jüngste Rückbestätigungsanfrage des Auftrags.

Ist die Anfrage weiterhin aktuell, wird das Callback-Ziel aus der dem Auftrag zugeordneten **Company** bestimmt. Dadurch kann für jeden Mandanten ein eigener DISPO-Endpunkt konfiguriert werden. Übertragen werden die externe Auftrags-ID, der Rückbestätigungsstatus und, sofern vorhanden, ein vom Kunden übermittelter Kommentar. Die ursprünglich aus DISPO übernommene externe Auftrags-ID ermöglicht die Zuordnung zur dortigen Auftragsidentität, ohne dass interne Datenbankidentifikatoren des Avisierungsservice über die Systemgrenze übertragen werden.

Wird die Rückmeldung aufgrund einer inzwischen neueren Anfrage unterdrückt, endet der Prozesspfad regulär ohne externen HTTP-Aufruf. Nach erfolgreicher Statusübertragung erreicht die Prozessinstanz ebenfalls ihr abschließendes Endereignis.

5.2.6 Fehlerbehandlung, Wiederholungen, Nebenläufigkeit und Idempotenzgrenzen

Neben dem regulären Prozessablauf muss der Avisierungsservice auch fehlgeschlagene, wiederholte und gleichzeitig ausgeführte Verarbeitungsschritte berücksichtigen. Hierfür werden unterschiedliche Mechanismen eingesetzt: Wiederholungen innerhalb des Workflows, technische Wiederholungsversuche durch die Workflow Engine, Datenbanksperren sowie Prüfungen des bereits erreichten Zustands.

Beim Nachrichtenversand werden Fehler danach unterschieden, ob ein weiterer Versandversuch sinnvoll vorgesehen ist oder der Versand unmittelbar als fehlgeschlagen behandelt wird. Vom Kommunikationssystem gemeldete Zustellfehler werden grundsätzlich als wiederholbar

behandelt. Eine feinere Unterscheidung zwischen kurzfristigen und dauerhaft bestehenden Ursachen erfolgt dabei nicht. Nicht wiederholbar sind dagegen beispielsweise eine fehlende notwendige E-Mail-Konfiguration oder der Fall, dass ein bei der Auftragsübernahme noch gültiges Lieferfenster zum tatsächlichen Versandzeitpunkt bereits begonnen hat.

Für wiederholbare Versandfehler sind maximal drei Versandversuche vorgesehen. Nach dem ersten fehlgeschlagenen Versuch wartet der Prozess eine Minute, vor dem dritten Versuch fünf Minuten. Für diese konkrete Anzahl und die Wartezeiten ist keine fachliche Herleitung dokumentiert; sie werden daher lediglich als Bestandteil der aktuellen technischen Konfiguration betrachtet. Kann die Nachricht nicht erfolgreich versendet werden, wird die Rückbestätigungsanfrage als **FAILED** markiert. Der Auftrag bleibt im Status **OPEN**, sodass eine erneute Avisierung möglich ist. Ein Versandfehler wird damit nicht mit einer Ablehnung oder einer ausgebliebenen Kundenreaktion gleichgesetzt.

Von diesen im Workflow vorgesehenen Versandwiederholungen sind unerwartete technische Fehler der Prozessausführung zu unterscheiden. Schlägt ein asynchroner Prozessschritt technisch fehl, versucht die Workflow Engine dessen Ausführung entsprechend der hinterlegten Konfiguration erneut. Für Versand und Fehlerverarbeitung sind jeweils drei Ausführungsversuche im Abstand von fünf Minuten vorgesehen, für die Statusrückmeldung an DISPO fünf Versuche im Abstand von einer Minute. Sind diese Versuche ausgeschöpft, erzeugt die Workflow Engine einen Incident. Eine eigene Incident-Verarbeitung wurde nicht implementiert; der Fehler muss daher operativ geprüft und behoben werden, bevor die betroffene Verarbeitung erneut ausgeführt werden kann.

Tabelle 1 fasst die Behandlung der unterschiedlichen Fehlerarten zusammen.

Tabelle 1: Behandlung unterschiedlicher Fehlerarten

Fehlerart	Behandlung	Folge bei dauerhaftem Fehler
Wiederholbarer Versandfehler	erneuter Versand innerhalb des Workflows	FAILED nach Ausschöpfung der Versandversuche
Nicht wiederholbarer Versandfehler	keine weitere Versandwiederholung	unmittelbarer Übergang in den Fehlerpfad
Unerwarteter technischer Prozessfehler	Job-Retry der Workflow Engine	Incident nach Ausschöpfung der Ausführungsversuche
Fehler beim DISPO-Callback	Job-Retry der Workflow Engine	erneuter Übertragungsversuch, ggf. Incident

Neben Fehlern müssen auch gleichzeitig eintretende Ereignisse berücksichtigt werden. Beispielsweise können eine Kundenreaktion und der Ablauf der Antwortfrist nahezu gleichzeitig verarbeitet werden. Für bereits vorhandene Aufträge wird deshalb eine pessimistische Datenbanksperre eingesetzt. Änderungen desselben Auftrags werden dadurch nacheinander verarbeitet, sodass ein späterer Verarbeitungsschritt den inzwischen aktualisierten Zustand berücksichtigt.

Eine Grenze besteht bei der erstmaligen parallelen Anlage eines Auftrags. Zu diesem Zeitpunkt existiert noch kein Datensatz, der gesperrt werden könnte. Eine Eindeutigkeitsbedingung auf Mandant und externe Auftrags-ID verhindert zwar, dass derselbe Auftrag dauerhaft doppelt gespeichert wird, garantiert jedoch nicht, dass beide gleichzeitig eintreffenden Aufrufe identisch erfolgreich verarbeitet werden.

Zusätzlich begrenzen Zustandsprüfungen unnötige Wiederholungen. Eine bereits erfolgreich versendete Rückbestätigungsanfrage wird bei einer erneuten Ausführung nicht nochmals versendet. Ebenso wird ein bereits verarbeiteter Versandfehler nicht erneut als neuer Zustandswechsel behandelt. Für eine Rückbestätigungsanfrage kann außerdem höchstens eine Kundenreaktion gespeichert werden. Vor einer Statusrückmeldung an DISPO wird zusätzlich geprüft, ob die zugehörige Anfrage weiterhin die jüngste Anfrage des Auftrags ist. Bereits erkennbare veraltete Rückmeldungen können dadurch unterdrückt werden.

Diese Mechanismen können jedoch keine vollständige Exactly-once-Ausführung über die Grenzen des Avisierungsservice hinweg sicherstellen. Beim Nachrichtenversand erfolgt zunächst der externe Aufruf und anschließend die Speicherung des erfolgreichen Versands. Wird die lokale Verarbeitung nach einem bereits erfolgreichen externen Aufruf unterbrochen, kann bei einer erneuten Ausführung nicht vollständig ausgeschlossen werden, dass die Nachricht nochmals versendet wird. Eine ähnliche Grenze besteht beim DISPO-Callback, da die Aktualitätsprüfung und der anschließende HTTP-Aufruf keine gemeinsame atomare Operation bilden.

Die Kombination aus Wiederholungen, Datenbanksperrern und Zustandsprüfungen reduziert damit fehlerhafte, konkurrierende und doppelte Verarbeitung. An den Grenzen lokaler Transaktionen und externer Systeme verbleiben jedoch Fehlerfenster, in denen keine globale Exactly-once-Ausführung gewährleistet werden kann.

5.3 Kanalunabhängige Kundenkommunikation

Die ausgehende Kundenkommunikation übermittelt die im Avisierungsprozess ausgelösten Nachrichten an den Kunden. Die initiale Rückbestätigungsanfrage kann über E-Mail, SMS oder WhatsApp versendet werden. Zusätzlich ist nach einer verarbeiteten Kundenreaktion eine Eingangsbestätigung vorgesehen.

Die Kanalunabhängigkeit bezieht sich auf die Auslösung der Kommunikationsvorgänge durch die Anwendungslogik. Nachrichtenerzeugung und technische Übertragung bleiben dagegen kanalspezifisch. Im Folgenden wird zunächst die Kommunikationsabstraktion und Kanalauswahl erläutert. Anschließend werden der E-Mail-Versand sowie die Anbindung von SMS und WhatsApp betrachtet.

5.3.1 Kommunikationsabstraktion und Kanalauswahl

Um die fachliche Prozesslogik von konkreten Übertragungstechnologien zu entkoppeln, verwendet der Application Layer den `NotificationService` als ausgehende Kommunikationsgrenze. Darüber werden die initiale Rückbestätigungsanfrage sowie die Eingangsbestätigung nach einer Kundenreaktion ausgelöst, ohne dass die aufrufenden Application Services von SMTP- oder Provider-Schnittstellen abhängen.

Der für eine konkrete Rückbestätigungsanfrage zu verwendende Kommunikationskanal ist im zugehörigen `ConfirmationRequest` festgelegt. Die Auswahl der konkreten Kanalimplementierung ist nach dem Strategy Pattern strukturiert (**gamma1994designPatterns**). Für E-Mail, SMS und WhatsApp existieren getrennte Sender-Implementierungen mit einem gemeinsamen Vertrag. Eine zentrale Kommunikationskomponente wählt zur Laufzeit anhand des gespeicherten `CommunicationChannel` die passende Strategie aus und delegiert den Kommunikationsvorgang an diese. Dadurch bleibt die Kanalauswahl an einer Stelle

gebündelt, während die kanalspezifische Umsetzung in den jeweiligen Sendern gekapselt ist.

Die laufzeitbezogene Kanalauswahl ist in Abbildung 6 dargestellt.

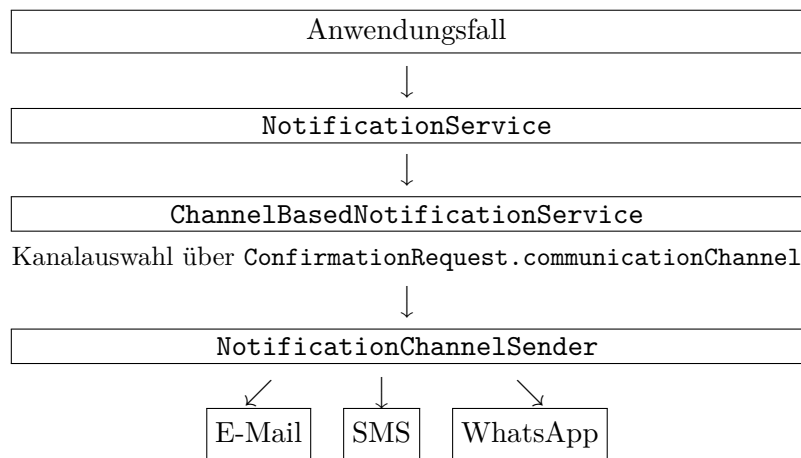


Abbildung 6: Laufzeitbezogene Kanalauswahl nach dem Strategy Pattern

Für sämtliche Kommunikationskanäle existiert kein einheitliches Nachrichtenmodell. Stattdessen erhalten die jeweiligen Sender die für den Kommunikationsvorgang erforderlichen fachlichen Daten aus `Order` und `ConfirmationRequest`; bei der Eingangsbestätigung wird zusätzlich die Kundenreaktion berücksichtigt. Die konkrete Nachricht wird anschließend innerhalb der jeweiligen Kanalimplementierung aufbereitet. Dadurch können die unterschiedlichen Anforderungen von E-Mail, SMS und WhatsApp berücksichtigt werden, ohne die Anwendungslogik an deren jeweilige Nachrichtenformate zu koppeln. Gleichzeitig greifen die Sender bei der Nachrichtenerzeugung direkt auf die übergebenen Domänenobjekte zu, sodass Änderungen an diesen Datenstrukturen auch Anpassungen in den Kommunikationsadaptern erforderlich machen können.

Technische Fehler des Versandvorgangs werden gegenüber der Anwendungslogik über die gemeinsame `NotificationDeliveryException` repräsentiert. Sie enthält zusätzlich den betroffenen Kommunikationskanal und verhindert damit, dass die Application Services unmittelbar von bibliotheks- oder providerspezifischen Fehlertypen abhängen. Wie ein solcher Fehler weiterbehandelt wird, hängt vom jeweiligen Anwendungsfall ab und wurde in den Abschnitten 5.2.3 und 5.2.6 beschrieben.

Der gemeinsame Sender-Vertrag sieht neben der initialen Rückbestätigungsanfrage auch eine Eingangsbestätigung nach einer Kundenreaktion vor. Im aktuellen Projektstand wird diese Eingangsbestätigung jedoch nur über E-Mail versendet. Für SMS und WhatsApp wurde sie aufgrund technischer Einschränkungen des im Projekt verwendeten Twilio-Demo-Setups nicht umgesetzt. Die initiale Rückbestätigungsanfrage wird dagegen über alle drei vorgesehenen Kommunikationskanäle unterstützt.

Die zentrale Auswahl- und Delegationslogik muss bei der Ergänzung eines weiteren Kommunikationskanals nicht um eine zusätzliche kanalspezifische Verzweigung erweitert werden. Neue Kanäle können dennoch Anpassungen an Adapter, Konfiguration oder Kontaktvalidierung erfordern. Die folgenden Abschnitte konkretisieren die technische Umsetzung zunächst für E-Mail und anschließend für die über Twilio angebotenen SMS- und WhatsApp-Kanäle.

5.3.2 E-Mail-Versand und mandantenspezifische SMTP-Konfiguration

Nach der in Abschnitt 5.3.1 beschriebenen Kanalauswahl übernimmt der E-Mail-Adapter die kanalspezifische Erzeugung und Übertragung der Nachricht. Dabei werden die Aufbereitung des Nachrichteninhalts und die technische Übertragung über SMTP voneinander getrennt.

Für die HTML-Aufbereitung werden drei Thymeleaf-Templates verwendet: eines für die initiale Rückbestätigungsanfrage sowie jeweils eines für die Eingangsbestätigung nach einer Bestätigung oder Ablehnung durch den Kunden. Die erforderlichen Auftrags- und Lieferdaten stammen aus `Order` und `ConfirmationRequest`. Zusätzlich wird für alle drei Nachrichtenvarianten aus der konfigurierten Frontend-Adresse und dem Token der Rückbestätigungsanfrage eine individuelle URL erzeugt. Nachrichtendarstellung und SMTP-Transport werden damit als getrennte technische Verantwortlichkeiten behandelt.

Abbildung 7 zeigt das Ergebnis dieser HTML-Aufbereitung anhand einer mit anonymisierten Testdaten erzeugten Rückbestätigungs-E-Mail.

Da SMTP-Infrastruktur und Absenderidentität mandantenbezogen konfigurierbar sind, wird kein globaler Mail-Sender für alle Unternehmen verwendet. Für einen konkreten Versand wird über den Auftrag die zugehörige `Company` bestimmt und deren SMTP-Konfiguration geladen. Diese umfasst unter anderem SMTP-Host und -Port, Sicherheitsmodus, optionale Authentifizierung sowie Absenderadresse und Absendername. Die Einstellungen werden somit mandantenbezogen gespeichert und nicht als globale Anwendungskonfiguration geführt.

Auf Grundlage dieser Einstellungen wird zur Laufzeit ein Mail-Sender für den jeweiligen Mandanten konfiguriert. Unterstützt werden die Sicherheitsmodi `STARTTLS`, `IMPLICIT_TLS` und `NONE`. Bei aktivierter SMTP-Authentifizierung werden zusätzlich Benutzername und Kennwort berücksichtigt. Das gespeicherte Kennwort wird erst für den Aufbau der Verbindung entschlüsselt; seine kryptografische Absicherung wird in Abschnitt 5.4.5 behandelt. Die dynamische Konfiguration ermöglicht die Verwendung unterschiedlicher SMTP-Einstellungen je Mandant, erfordert jedoch den erneuten Aufbau des Mail-Senders für den jeweiligen Versand.

Anschließend werden der zuvor erzeugte HTML-Inhalt und die mandantenspezifische SMTP-Konfiguration für den Versand zusammengeführt. Als Absender dienen die hinterlegte Adresse und Bezeichnung des Mandanten, als Empfänger die im Auftrag gespeicherte E-Mail-Adresse des Kunden. Die Nachricht enthält Betreff, HTML-Inhalt und ein eingebettetes Logo und wird anschließend an den konfigurierten SMTP-Server übergeben. Fehlt für den Mandanten eine SMTP-Konfiguration, kann kein Versand erfolgen. Technische Fehler bei der Vorbereitung oder Übertragung der Nachricht werden gegenüber der Anwendungslogik über eine gemeinsame Versandfehlerabstraktion dargestellt.

Die SMTP-Einstellungen können mandantenbezogen gelesen und aktualisiert werden. Ein gespeichertes Kennwort wird dabei nicht an den Client zurückgegeben; stattdessen zeigt `passwordConfigured` lediglich an, ob ein Kennwort vorhanden ist. Wird bei aktivierter Authentifizierung kein neues Kennwort übermittelt, bleibt das bestehende erhalten. Dadurch können die übrigen SMTP-Einstellungen geändert werden, ohne das gespeicherte Kennwort erneut übertragen zu müssen.

Wird die SMTP-Authentifizierung deaktiviert, werden die dafür nicht mehr benötigten

Lieferfenster zur Bestätigung

MINOVA

Guten Tag Herr/Frau **Daniel Tempel**,
bitte prüfen Sie das vorgeschlagene Lieferfenster für Ihre Heizöl-Lieferung.

VORGESCHLAGENER LIEFERTERMIN

31.08.2026

18:00 - 20:00 Uhr

Lieferfenster beantworten

Auftragsdaten	
Adresse	Valentin-Becker-Strasse, 97072 Würzburg
Produkt	Heizöl EL
Menge	3000 Liter
Preis	95,00 EUR / 100 L

Wenn das Zeitfenster nicht passt, können Sie dies über die Antwortseite mitteilen.

Viele Grüße
Ihr Serviceteam

Diese E-Mail wurde automatisch erstellt. Bitte antworten Sie nicht direkt auf diese Nachricht.

Abbildung 7: Beispiel einer durch den Avisierungsservice erzeugten Rückbestätigungs-E-Mail

Quelle: Eigene Darstellung.

Zugangsdaten aus der Konfiguration entfernt. Ein Betrieb ohne Authentifizierung und ohne Transportverschlüsselung ist im Projekt jedoch ausschließlich für die lokale Entwicklungsumgebung vorgesehen, in der mit einem lokalen Mailserver gearbeitet wird. Für einen produktiven Einsatz ist eine authentifizierte und transportgesicherte SMTP-Verbindung vorgesehen. Die bereitgestellte Entwicklungsumgebung verwendet entsprechend einen lokalen SMTP-Server ohne Authentifizierung.

Zur technischen Prüfung stehen ein Verbindungstest und ein separater Testversand zur Verfügung. Der Verbindungstest verwendet dieselbe mandantenspezifische Konfiguration wie der reguläre Versand und prüft, ob mit den gespeicherten Verbindungs- und gegebenenfalls Authentifizierungsdaten eine Verbindung zum SMTP-System hergestellt werden kann.

Der Testversand führt mit derselben Transportkonfiguration eine tatsächliche SMTP-Sendeoperation aus und sendet eine einfache Testnachricht an die konfigurierte Absenderadresse. Die fachliche Erzeugung einer Rückbestätigungs-E-Mail mit Thymeleaf-Template, Auftragsdaten und Bestätigungslink ist davon nicht umfasst. Ein erfolgreicher Testversand zeigt daher lediglich, dass der SMTP-Server bei der Übertragung keinen Fehler gemeldet hat; eine endgültige Zustellung in das Postfach des Empfängers wird damit nicht nachgewiesen.

Die E-Mail-Implementierung verbindet somit eine getrennte HTML-Aufbereitung mit einer zur Laufzeit bestimmten mandantenspezifischen SMTP-Konfiguration. Die in Abschnitt 5.3.3 betrachteten Kanäle SMS und WhatsApp verwenden dagegen eine externe Provider-Anbindung.

5.3.3 SMS und WhatsApp über Twilio: Technologieauswahl und Integration

5.4 Mandantenfähigkeit und Sicherheitskonzept

Der Avisierungsservice ist als mandantenfähige gemeinsame Anwendung konzipiert. Mehrere voneinander unabhängige Unternehmen nutzen dieselbe fachliche Funktionalität und eine einheitlich definierte DISPO-Schnittstelle. Auftrags- und Kundendaten sowie mandantenspezifische Konfigurationen müssen dabei eindeutig einem Unternehmen zugeordnet und voneinander getrennt verarbeitet werden.

Aus den unterschiedlichen Zugriffs- und Vertrauenskontexten ergeben sich mehrere Schutzbereiche mit jeweils eigenen Zugriffsmodellen. Die Zuordnung dieser Schutzbereiche zu den nachfolgenden Abschnitten ist in Tabelle 2 zusammengefasst.

Zunächst wird betrachtet, wie die Mandantenidentität innerhalb der Anwendung repräsentiert und für die logische Trennung gemeinsam gespeicherter Daten verwendet wird.

5.4.1 Mandantenkontext und Datenisolation

Für die Datenhaltung wurde bewusst eine gemeinsame Datenbank mit gemeinsamem Schema gewählt. Separate Datenbanken ermöglichen eine stärkere physische Isolation, getrennte Schemas eine stärkere strukturelle Trennung. Beide Varianten erhöhen jedoch den mandantenspezifischen Provisionierungs-, Migrations- und Betriebsaufwand. Das

Tabelle 2: Schutzbereiche des Mandanten- und Sicherheitskonzepts

Schutzbereich	Sicherheitsaufgabe	Abschnitt
DISPO-Schnittstelle	externes System authentifizieren und Mandantenidentität bestimmen	5.4.2
Dashboard-Zugriff	Browserzugriff an einen authentifizierten Mandantenkontext binden	5.4.3
Kundenzugriff	Zugriff auf eine konkrete Rückbestätigungsanfrage begrenzen	5.4.4
Mandantenspezifische Konfigurationsgeheimnisse	sensible technische Konfigurationsdaten vertraulich speichern	5.4.5

realisierte Modell lässt sich als Pool-Modell einordnen, bei dem mehrere Mandanten dieselben Speicherstrukturen nutzen und ihre Daten logisch über eine Mandantenidentität getrennt werden (**aws2021partitioningModels**). Da für die angebundenen Unternehmen dieselbe Fachlogik und Datenstruktur gelten, wurde für den Projektumfang der geringere infrastrukturelle Aufwand gegenüber einer stärkeren Isolation priorisiert.

Zur Laufzeit wird eine bereits bestimmte Mandantenidentität durch den **CompanyContext** repräsentiert, der ausschließlich die interne **companyId** enthält. Dadurch müssen die mandantenbezogenen Anwendungsfälle der DISPO-, Dashboard- und Konfigurationsschnittstellen den vorgelagerten Authentifizierungsmechanismus nicht kennen. Die Mandantenidentität wird nicht aus frei übermittelbaren fachlichen Request-Daten übernommen, sondern zuvor durch den jeweiligen Sicherheitsmechanismus bestimmt und anschließend getrennt von den Nutzdaten an die Anwendungslogik übergeben. Die konkrete Ermittlung des **CompanyContext** für die DISPO-Schnittstelle wird in Abschnitt 5.4.2 erläutert.

Der **CompanyContext** erzeugt für sich genommen noch keine Datenisolation. Entscheidend ist seine Verwendung bei den nachfolgenden Datenzugriffen. So wird ein bestehender DISPO-Auftrag anhand der Kombination aus **companyId** und externer Auftrags-ID gesucht. Dadurch kann dieselbe externe Auftrags-ID bei verschiedenen Unternehmen vorkommen, während sie innerhalb eines Mandanten eindeutig bleibt.

Auch Dashboard-Abfragen werden auf die im **CompanyContext** enthaltene **companyId** eingeschränkt. Die Detailansicht bestimmt zunächst den Auftrag im Kontext des Mandanten und lädt anschließend die zugehörigen Rückbestätigungsanfragen und Kundenreaktionen. Mandantenspezifische technische Konfigurationen folgen demselben Prinzip; beispielsweise werden die in Abschnitt 5.3.2 beschriebenen SMTP-Einstellungen über die zugehörige **Company** bestimmt.

Datenbankseitige Fremdschlüssel- und Eindeutigkeitsbedingungen stellen die konsistente Zuordnung von Datensätzen zu einer **Company** sicher, ersetzen jedoch keine Zugriffsbeschränkung. Eine fehlerhaft formulierte Abfrage könnte trotz korrekter Datenzuordnung Datensätze eines anderen Mandanten berücksichtigen. Mandantentrennung ist daher von der bloßen Authentifizierung zu unterscheiden: Auch ein authentifizierter Zugriff muss auf die Ressourcen des zugehörigen Mandanten beschränkt werden (**aws2022tenantIsolation**).

Nicht alle Verarbeitungspfade verwenden den **CompanyContext**. Interne Workflow-Schritte arbeiten mit bereits persistierten technischen Objektidentifikatoren und den darüber erreichbaren fachlichen Beziehungen. Der Kundenzugriff verwendet dagegen einen begrenzten

tokenbasierten Zugriff auf eine konkrete Rückbestätigungsanfrage, der in Abschnitt 5.4.4 behandelt wird.

Eine zusätzliche datenbankseitige Zugriffskontrolle durch Row-Level Security oder einen globalen ORM-Tenant-Filter wird nicht eingesetzt; neue mandantenbezogene Datenzugriffe müssen den jeweiligen Mandantenbezug daher explizit berücksichtigen.

Ein Laufzeit-Onboarding neuer Mandanten ist im aktuellen Prototyp nicht umgesetzt. Der initiale **Company**-Datensatz wird über die Datenbankmigration bereitgestellt; ein festgelegter Anwendungsprozess zur Aufnahme weiterer Mandanten besteht nicht. Das Datenmodell selbst ist jedoch nicht auf einen einzelnen Mandanten beschränkt.

Die Mandantentrennung entsteht somit durch das Zusammenspiel aus einer vertrauenswürdig bestimmten Mandantenidentität, ihrer Weitergabe als **CompanyContext**, mandantenbezogenen Datenzugriffen und datenbankseitiger Zuordnungsintegrität. Wie die Mandantenidentität bei einem Zugriff durch DISPO ermittelt wird, beschreibt der folgende Abschnitt.

5.4.2 Authentifizierung der DISPO-Schnittstelle

Für die System-zu-System-Kommunikation wurde eine mandantenspezifische API-Key-Authentifizierung gewählt. Sie ermöglicht die direkte Zuordnung des aufrufenden externen Systems DISPO zu einer **Company**, ohne zusätzliche Identity-Provider-Infrastruktur zu benötigen. Die Pfade unter `/api/dispo/**` bilden hierfür einen eigenen zustandslosen Schutzbereich. Der API-Key wird bei jeder HTTP-Anfrage im Header **X-API-Key** übermittelt und erneut geprüft. Dem vergleichsweise geringen Integrationsaufwand steht die Verwendung eines wiederverwendbaren Secrets gegenüber, dessen Schutz und Lebenszyklus gesondert berücksichtigt werden müssen.

Zur Authentifizierung wird der empfangene API-Key mit SHA-256 gehasht und der resultierende Wert mit den gespeicherten API-Key-Hashes der **Company**-Datensätze abgeglichen. Bei erfolgreicher Zuordnung wird aus der internen **Company.id** ein **CompanyContext** erzeugt und als Principal des authentifizierten Aufrufs verwendet. Die fachliche Verarbeitung erhält damit ausschließlich den bestimmten Mandantenkontext und nicht den Rohschlüssel. Fehlt der API-Key oder kann sein Hash keiner **Company** zugeordnet werden, wird die Anfrage vor der fachlichen Verarbeitung mit **401 Unauthorized** zurückgewiesen.

Der API-Key wird nicht im Klartext in der Anwendungsdatenbank persistiert; jeder **Company** ist ein eindeutiger **apiKeyHash** zugeordnet. Eine Offenlegung der Datenbank legt den Rohschlüssel dadurch nicht unmittelbar offen. Da der verwendete SHA-256-Hash jedoch eine Offline-Prüfung möglicher Schlüsselwerte erlaubt, müssen die API-Keys kryptografisch zufällig und mit ausreichender Entropie erzeugt werden (**owaspSecretsManagement**).

Die Verwaltung des Schlüssellebenszyklus erfolgt im aktuellen Prototyp außerhalb der Anwendung. Eine Änderung oder ein Widerruf erfordert den administrativen Austausch des gespeicherten Hashwertes; automatische Rotation, Ablaufzeiten oder die parallele Gültigkeit mehrerer Schlüssel sind nicht implementiert.

Von der Authentifizierung ist die anschließende Autorisierung zu unterscheiden. Ein gültiger API-Key authentifiziert die zugeordnete **Company**. Die aktuelle Autorisierungsregel gewährt jedem so authentifizierten Principal Zugriff auf sämtliche Endpunkte unter `/api/dispo/**`. Eine weitere Differenzierung nach Rollen, Scopes oder endpointbezogenen Berechtigun-

gen ist nicht implementiert. Damit besteht eine technische Identität pro **Company** mit einheitlichem Berechtigungsumfang.

Die beschriebene Authentifizierung betrifft ausschließlich eingehende DISPO-Aufrufe. Die in Abschnitt 5.2.5 beschriebene ausgehende Statusrückmeldung enthält im aktuellen Prototyp keinen zusätzlichen anwendungsseitigen Authentifizierungsnachweis. Der darauf aufbauende Browserzugriff auf das Avisierungsdashboard verwendet einen separaten Sicherheitsmechanismus, der im folgenden Abschnitt behandelt wird.

5.4.3 Zugriffsschutz des Avisierungsdashboards

Der Zugriff auf das Avisierungsdashboard bildet einen anderen Schutzbereich als die in Abschnitt 5.4.2 beschriebene System-zu-System-Kommunikation. Als Vertrauensannahme gilt, dass DISPO den jeweiligen Disponenten bereits authentifiziert hat. Der Avisierungsservice überprüft diese personenbezogene Identität nicht selbst, sondern bestimmt über die authentifizierte DISPO-Schnittstelle ausschließlich den zugehörigen Mandanten. Der DISPO-API-Key verbleibt innerhalb der System-zu-System-Kommunikation und wird nicht an den Browser weitergegeben. Für den Dashboardzugriff wird somit die bereits bestimmte Mandantenidentität übertragen, nicht das langfristig gültige Authentifizierungsgeheimnis.

Fordert DISPO einen Dashboardzugriff an, erzeugt der Avisierungsservice einen kurzlebigen Einmalcode für die im **CompanyContext** enthaltene **companyId**. Der Code basiert auf 32 kryptografisch zufälligen Bytes, wird serverseitig der Mandantenidentität zugeordnet und ist eine Minute gültig. DISPO erhält eine Frontend-URL mit dem Code als Query-Parameter und öffnet diese im vorgesehenen Integrationsablauf im Browser des Disponenten. Der Avisierungsservice selbst führt keine HTTP-Weiterleitung aus.

Der Besitz eines gültigen Codes berechtigt einmalig zum Aufbau einer Dashboard-Sitzung für den zugeordneten Mandanten; eine Bindung an einen bestimmten Browser oder Disponenten besteht nicht. Bis zu seiner Verwendung stellt der Code daher einen kurzlebigen, besitzgebundenen Berechtigungsnachweis dar. Da er über einen URL-Parameter übertragen wird, kann er beispielsweise in der Browserhistorie oder in Protokolldaten sichtbar werden. Die kurze Gültigkeit und die einmalige Verwendbarkeit begrenzen dieses Risiko, beseitigen es jedoch nicht vollständig.

Das Avisierungsdashboard übermittelt den Code anschließend an den Exchange-Endpunkt des Avisierungsservice. Beim Austausch wird der Eintrag aus dem In-Memory-Speicher entfernt und seine zeitliche Gültigkeit geprüft. Aus der gespeicherten Mandantenzuordnung wird ein **CompanyContext** wiederhergestellt, als authentifizierter Principal in den Spring-Security-Kontext übernommen und in einer serverseitigen HTTP-Sitzung gespeichert. Für die Authentifizierung weiterer Dashboardzugriffe verwendet der Browser ausschließlich die vom Avisierungsservice ausgestellte Session-ID im Cookie; bei zustandsändernden Anfragen kommt zusätzlich ein CSRF-Token zum Einsatz.

Das Session-Cookie ist mit **HttpOnly** und **SameSite=Lax** konfiguriert; im Produktivprofil wird zusätzlich **Secure** verwendet. **SameSite=Lax** begrenzt die automatische Übermittlung des Cookies in bestimmten siteübergreifenden Kontexten, ersetzt jedoch keinen eigenständigen CSRF-Schutz. Besteht bereits eine HTTP-Sitzung, wird deren Kennung beim Aufbau der Authentifizierung erneuert, um die Übernahme einer zuvor bekannten Sitzungskennung zu vermeiden. Dies dient dem Schutz vor Session-Fixation-Angriffen.

Für die Sitzung ist eine Inaktivitätsdauer von acht Stunden konfiguriert. Sie beschreibt keine

absolute maximale Lebensdauer; die Sitzung wird ungültig, wenn innerhalb dieses Zeitraums keine ihr zugeordnete Anfrage eingeht. Eine zusätzliche absolute Laufzeitbegrenzung ist im Prototyp nicht vorgesehen.

Mit Ausnahme des Exchange-Endpunkts erfordern die Pfade unter `/api/dashboard/**` einen authentifizierten Security Context. Der darin gespeicherte **CompanyContext** wird den Dashboard-Anwendungsfällen als Mandantenkontext übergeben. Die Mandantenidentität wird somit nicht aus Request-Parametern oder fachlichen Nutzdaten bestimmt; die weitere Datenisolation erfolgt wie in Abschnitt 5.4.1 beschrieben.

Da der Browser das Session-Cookie automatisch mitsendet, werden zustandsändernde Dashboardzugriffe zusätzlich durch CSRF-Schutz abgesichert. Nach dem Sitzungsaufbau kann ein CSRF-Token abgerufen und im Header **X-CSRF-TOKEN** übertragen werden. Im gewählten Ablauf wird vor dem Austausch des Einmalcodes keine vorbereitende Dashboard-Sitzung mit CSRF-Token aufgebaut; der Exchange-Endpunkt ist daher in der aktuellen Implementierung vom regulären CSRF-Schutz ausgenommen. Der Einmalcode begrenzt diesen initialen Zugriff zeitlich und auf eine einmalige Verwendung, stellt jedoch keinen allgemeinen Schutz gegen Login-CSRF dar (**owaspCsrfPrevention**).

Ergänzend erlaubt die CORS-Konfiguration die Nutzung der Dashboard-Schnittstelle unter Einbeziehung von Cookies ausschließlich für die konfigurierte Frontend-Adresse und den benötigten CSRF-Header. CORS ersetzt dabei weder die Authentifizierung noch den CSRF-Schutz.

Abbildung 8 fasst den Übergang vom authentifizierten DISPO-Kontext zur browserbasierten Dashboard-Sitzung zusammen.

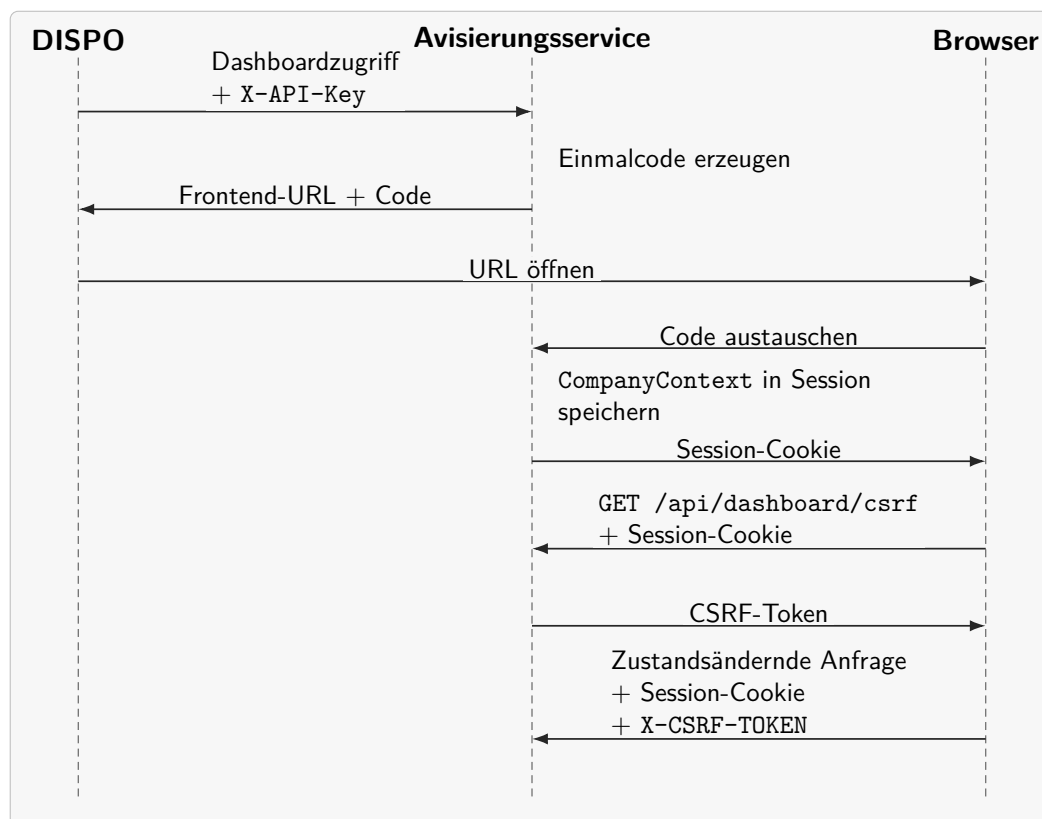


Abbildung 8: Übergang vom authentifizierten DISPO-Kontext zur Dashboard-Sitzung

Das Sicherheitsmodell besitzt bewusst akzeptierte Grenzen. Der Avisierungsservice authentifiziert ausschließlich die **Company**, nicht den einzelnen Disponenten; eigene Benutzerkonten, Rollen und personenbezogene Berechtigungen bestehen daher nicht. Die Einmalcodes werden nur im lokalen Arbeitsspeicher gespeichert, weshalb der Mechanismus auf eine einzelne Instanz des Avisierungsservice ausgelegt ist. Auch die serverseitige Sitzung macht den Betrieb zustandsbehaftet. Eine explizite Abmeldung wurde im Prototyp nicht umgesetzt; ungenutzte Sitzungen verlieren nach Ablauf der konfigurierten Inaktivitätsdauer ihre Gültigkeit.

Der davon getrennte Kundenzugriff wird im folgenden Abschnitt betrachtet.

5.4.4 Tokenbasierter Kundenzugriff

Der Kundenzugriff bildet einen eigenen Schutzbereich. Im Unterschied zur DISPO-Schnittstelle und zum Avisierungsdashboard verfügt der Kunde über keine eigene technische Identität innerhalb des Avisierungsservice. Es existieren weder Kundenkonten noch eine browserbasierte Sitzung oder ein **CompanyContext**. Der Zugriff auf die kundenbezogenen Funktionen erfolgt daher über einen zufällig erzeugten Zugriffstoken. Dieser authentifiziert keine Person, sondern ermöglicht dem Besitzer den Zugriff auf den damit verbundenen Kundenvorgang. Dadurch kann auf eine zusätzliche Registrierung oder Anmeldung des Kunden verzichtet werden; gleichzeitig muss der Token selbst vor unbefugter Weitergabe geschützt werden.

Bei der Neuanlage einer **ConfirmationRequest** wird ein neuer Token erzeugt. Hierzu werden mit **SecureRandom** 32 zufällige Bytes generiert und URL-kompatibel über **Base64** kodiert. Der daraus resultierende Zufallswert besitzt 256 Bit Entropie und erschwert das Erraten gültiger Tokens. Werden für einen Auftrag nacheinander mehrere Rückbestätigungsanfragen erzeugt, erhält jede neu angelegte Anfrage einen eigenen Token.

Der Token wird Bestandteil des individuellen Bestätigungslinks und über die in Abschnitt 5.3 beschriebenen Kommunikationskanäle an den Kunden übertragen. Über denselben Token kann anschließend die Bestätigungsseite aufgerufen, eine Kundenreaktion übermittelt und auf die für den Kunden vorgesehenen Funktionen der Lieferverfolgung zugegriffen werden. Die Zugriffsprüfung erfolgt anhand des bei jedem Aufruf übermittelten Tokens. Da dabei keine authentifizierte Browser-Sitzung verwendet wird, sind diese Endpunkte in der Spring-Security-Konfiguration von der CSRF-Prüfung ausgenommen.

Für die Auflösung eines Tokens reicht dessen Existenz in der Datenbank nicht aus. Er muss zusätzlich zur neuesten **ConfirmationRequest** des jeweiligen Auftrags gehören. Technisch wird dafür geprüft, ob die zugehörige Anfrage den höchsten Datenbankidentifikator unter den Rückbestätigungsanfragen desselben Auftrags besitzt. Wird nach einer Anfrage A eine neuere Anfrage B erzeugt, bleibt Token A zwar historisch gespeichert, kann über die Kundenschnittstelle jedoch nicht mehr aufgelöst werden. Dadurch verlieren ältere Kundenlinks ihre Gültigkeit, ohne dass die zugehörigen historischen Anfragen gelöscht werden müssen.

Welche Funktion nach erfolgreicher Tokenauflösung genutzt werden kann, hängt zusätzlich vom Zustand des Vorgangs ab. Die Bestätigungsseite bleibt auch nach Ablauf der Antwortfrist verfügbar, solange keine neuere Rückbestätigungsanfrage angelegt wurde. Dies ist bewusst vorgesehen, damit der Kunde den Liefertermin später erneut einsehen und denselben Link am Liefertag für die Funktionen der Lieferverfolgung verwenden kann. Eine nachträgliche Bestätigung oder Ablehnung ist dagegen nicht möglich, da hierfür zusätzlich

geprüft wird, ob die Anfrage noch aktiv, nicht abgelaufen und noch unbeantwortet ist.

Auch die Funktionen der Lieferverfolgung verwenden dieselbe Tokenauflösung. Informationen zur Lieferverfolgung werden nur an dem im **DeliverySlot** hinterlegten Lieferdatum als verfügbar ausgewiesen. Eine Fahrerposition wird zusätzlich nur dann bereitgestellt, wenn entsprechende Positionsdaten vorliegen. Die funktionale Ermittlung dieser Daten wird in Abschnitt 5.6 behandelt.

Der gewählte Zugriff über einen zufälligen Token besitzt dennoch Grenzen. Da keine Personenidentität überprüft wird, kann grundsätzlich jeder, der einen aktuellen Token erhält, die dafür freigegebenen Funktionen verwenden. Im aktuellen Prototyp wird der Token zudem direkt in der Datenbank gespeichert; eine hashbasierte Speicherung wäre als zusätzliche Härtnungsmaßnahme möglich.

Die bewusst beibehaltene Lesbarkeit der Bestätigungsseite verlängert außerdem den Zeitraum, in dem der jeweils letzte Token verwendbar bleibt. Solange keine neuere **ConfirmationRequest** entsteht, kann die Bestätigungsseite auch nach Antwortfrist und Liefertag weiterhin geöffnet werden. Dies ermöglicht die spätere Einsicht in Lieferinformationen, erhöht jedoch zugleich die Auswirkungen einer möglichen Offenlegung des Tokens.

Da der Token Bestandteil des URL-Pfades ist, kann eine Offenlegung beispielsweise über Browserhistorien, URL-Protokollierung oder Referer-Informationen erfolgen. Für den produktiven Betrieb wird daher eine durchgängige HTTPS-Übertragung vorausgesetzt. Diese schützt die Übertragung, verhindert jedoch nicht die spätere Offenlegung einer bereits bekannten oder protokollierten URL (**owaspInformationExposureUrls**).

Das Verfahren ermöglicht damit einen einfachen Kundenzugriff ohne zusätzliche Benutzerverwaltung. Die zufällige Token-Erzeugung und die Prüfung auf die jeweils neueste Rückbestätigungsanfrage erschweren das Erraten gültiger Tokens und schließen veraltete Links aus. Der Schutz des aktuell gültigen Tokens bleibt jedoch eine zentrale Voraussetzung des Zugriffsmodells.

5.4.5 Schutz sensibler Konfigurationsdaten

Das mandantenspezifische SMTP-Kennwort stellt ein wiederverwendbares technisches Zugangsgeheimnis dar. Da der Avisierungsservice den ursprünglichen Wert später selbst zur Authentifizierung gegenüber dem SMTP-System benötigt, reicht eine ausschließlich hashbasierte Speicherung, wie beim DISPO-API-Key, nicht aus. Die reversible Verschlüsselung ergibt sich somit aus dem konkreten technischen Verwendungszweck und ist keine Alternative zum Hashing von Benutzerpasswörtern.

Zur vertraulichen Speicherung und Erkennung nachträglicher Veränderungen wird AES-GCM mit einem 256-Bit-Schlüssel eingesetzt (**dworkin2007gcm**; **nist2023aes**). Für jeden Verschlüsselungsvorgang wird ein neuer zufälliger Initialisierungsvektor erzeugt. Zusätzlich wird die interne **companyId** als Additional Authenticated Data einbezogen. Dadurch ist das Chiffre kryptografisch an den vorgesehenen Mandantenkontext gebunden; eine Veränderung des Chiffres oder dieses Kontexts führt bei der Entschlüsselung zu einer fehlgeschlagenen Authentizitätsprüfung. Diese Bindung ergänzt die mandantenbezogene Zugriffskontrolle aus Abschnitt 5.4.1, ersetzt sie jedoch nicht.

Der verschlüsselte Passwortwert wird in der Anwendungsdatenbank gespeichert, während

der Master-Key über die externe Spring-Konfiguration bereitgestellt wird. Schlüsselmaterial und Chiffre sind damit logisch getrennt, sodass ein isolierter Zugriff auf die Datenbank das SMTP-Kennwort nicht unmittelbar offenlegt. Eine anwendungsseitige Integration eines externen Schlüssel- oder Secrets-Management-Systems ist im Prototyp nicht umgesetzt.

Ein neu konfiguriertes Kennwort wird über die mandantengebundene Konfigurationschnittstelle übermittelt und vor der Persistierung verschlüsselt. Für einen produktiven Betrieb setzt dieser Übertragungsweg eine transportgesicherte HTTPS-Verbindung voraus. Bereits gespeicherte Kennwörter werden, wie in Abschnitt 5.3.2 beschrieben, nicht erneut an den Client ausgegeben. Zur technischen SMTP-Nutzung wird das Kennwort beim Erzeugen des mandantenspezifischen Mail-Senders entschlüsselt, etwa für regulären Versand, Verbindungstest oder Testversand. Der Klartextwert muss daher zur Laufzeit innerhalb der Anwendung verarbeitet werden.

Die Lösung reduziert insbesondere das Risiko einer Offenlegung der SMTP-Zugangsdaten bei einem isolierten Zugriff auf die Datenbank. Da sämtliche mandantenspezifischen SMTP-Kennwörter denselben zentralen Master-Key verwenden, kann dessen Kompromittierung potenziell sämtliche damit geschützten Zugangsdaten betreffen; auch gegenüber einer vollständigen Kontrolle des laufenden Anwendungsprozesses bietet die Verschlüsselung keinen Schutz. Eine anwendungsseitige Schlüsselrotation ist im Prototyp ebenfalls nicht umgesetzt: Das Chiffre enthält zwar eine Formatversion, jedoch keinen Bezeichner des verwendeten Master-Keys, und mehrere gleichzeitig gültige Schlüssel oder eine automatische Neuverschlüsselung bestehender Werte werden nicht unterstützt. Ein bloßer Austausch des Master-Keys führt deshalb dazu, dass bereits gespeicherte Kennwörter nicht mehr entschlüsselt werden können.

5.5 Abfragemodell für das Avisierungsdashboard

Die in den vorhergehenden Abschnitten erzeugten Auftrags-, Avisierungs- und Rückbestätigungsdaten müssen der Disposition für die operative Bearbeitung bereitgestellt werden. Dazu bereitet der Avisierungsservice die gespeicherten Fach- und Prozessdaten entsprechend dem jeweiligen Informationsbedarf auf.

Das in Abschnitt 5.1.2 beschriebene Domänenmodell bildet die fachlichen Beziehungen und Zustände von `Order`, `ConfirmationRequest` und `CustomerResponse` ab. Für die dispositionsseitige Darstellung werden diese Daten jedoch anders strukturiert. Die persistierten Domänenobjekte werden daher nicht unmittelbar ausgegeben; stattdessen werden die für den jeweiligen Anwendungsfall benötigten Informationen gezielt zusammengestellt.

Das Abfragemodell unterscheidet hierzu zwei Sichten. Die Tourübersicht fasst die für die operative Bearbeitung relevanten Informationen mehrerer Aufträge im Kontext ihrer Touren zusammen und stellt dabei jeweils den aktuell maßgeblichen Avisierungsstand dar. Die auftragsbezogene Detailansicht stellt dagegen einen einzelnen Auftrag einschließlich seiner aktuellen und früheren Avisierungen bereit. Beide Sichten greifen damit auf dieselben gespeicherten Fach- und Prozessdaten zurück, unterscheiden sich jedoch in Umfang und Struktur der bereitgestellten Informationen.

Eine separate Lesedatenhaltung wird hierfür nicht verwendet. Die benötigten Ergebnissstrukturen werden bei der jeweiligen Abfrage aus derselben relationalen Datenbasis erzeugt, die auch der schreibenden Verarbeitung zugrunde liegt. Im Folgenden werden zunächst die

Tourübersicht mit dynamischer Filterung, Gruppierung und Paginierung und anschließend die auftragsbezogene Detailansicht mit Avisierungshistorie betrachtet.

5.5.1 Tourübersicht, dynamische Filterung, Gruppierung und Paginierung

Die Tourübersicht stellt die für die operative Bearbeitung relevanten Auftrags- und Avisierungsinformationen in einer tourbezogenen Struktur bereit. Eine Tour enthält dabei ihre tourbezogenen Angaben sowie eine reduzierte Sicht auf die zugehörigen Aufträge.

Da zu einem `Order` mehrere `ConfirmationRequest`-Objekte existieren können, berücksichtigt die Übersicht je Auftrag ausschließlich die Anfrage mit der höchsten internen ID. Frühere Rückbestätigungsanfragen bleiben für die Avisierungshistorie erhalten, werden jedoch nicht zusätzlich in der aktuellen Tourübersicht dargestellt.

Jede Abfrage wird zunächst auf die `companyId` des aktuellen Mandanten eingeschränkt. Zusätzlich können Tournummern, fachliche `ConfirmationStatus`, ein Datumsbereich und eine Freitextsuche miteinander kombiniert werden. Der Statusfilter bezieht sich auf den fachlichen Bearbeitungsstand des Auftrags und nicht auf den technischen Versandstatus einer einzelnen Rückbestätigungsanfrage. Wird kein Startdatum angegeben, dient das aktuelle Datum als Untergrenze; ungültige Datumsbereiche werden zurückgewiesen. Die Freitextsuche berücksichtigt ohne Unterscheidung von Groß- und Kleinschreibung Tournummer, Fahrzeugkennzeichen, externe Auftrags-ID, Kundenname und Lieferadresse.

Für die dynamische Abfragekonstruktion wird QueryDSL in seiner JPA-Integration eingesetzt. Die Bibliothek ermöglicht den typisierten Aufbau und die schrittweise Zusammensetzung von Abfragebedingungen (`querydslUnifiedQueries`). Dadurch können nur die tatsächlich gesetzten Filter berücksichtigt und mit der Auswahl der jeweils neuesten Rückbestätigungsanfrage, Gruppierung, Sortierung und Paginierung kombiniert werden. Separate statische Abfragen für jede mögliche Filterkombination sind dadurch nicht erforderlich.

Da die Tour die zentrale Einheit der Übersicht bildet, erfolgt auch die Paginierung auf Tour-Ebene und nicht auf einzelnen Aufträgen. Zunächst werden daher die Touren der angeforderten Seite bestimmt und ihre Gesamtzahl für die Paginierung ermittelt. Anschließend werden die zu diesen Touren gehörenden Auftragsinformationen geladen. Dadurch beeinflusst die unterschiedliche Anzahl von Aufträgen innerhalb einer Tour nicht die Einteilung der Tourseiten.

Die Filterbedingungen werden sowohl bei der Auswahl der Touren als auch beim Laden ihrer Aufträge berücksichtigt. Bei auftragsbezogenen Filtern enthält eine zurückgegebene Tour daher nur diejenigen Aufträge, die die gewählten Bedingungen erfüllen. Die Touren werden nach Lieferdatum und Tournummer sortiert, die enthaltenen Aufträge nach Beginn und Ende des Lieferfensters sowie nach externer Auftrags-ID.

Für die Filterauswahl wird ergänzend eine reduzierte Abfrage der im aktuellen Mandanten- und Datumsbereich verfügbaren Tournummern bereitgestellt. Sie verwendet dieselben grundlegenden Mandanten-, Datums- und Aktualitätsbedingungen wie die Tourübersicht, beschränkt die Suchfunktion jedoch auf die Tournummer. Während die Tourübersicht damit den aktuell maßgeblichen Bearbeitungsstand darstellt, behandelt Abschnitt 5.5.2 zusätzlich die einzelnen Avisierungsversuche und deren Historie.

5.5.2 Auftragsdetails und Avisierungshistorie

Während die Tourübersicht mehrere Aufträge in reduzierter Form darstellt, bündelt die Detailansicht die Informationen zu einem einzelnen Auftrag. Dieser wird innerhalb des bestehenden Mandantenkontexts über `companyId` und externe Auftrags-ID ermittelt. Angezeigt werden die aktuellen Auftragsdaten einschließlich Kunden- und Kontaktdaten, Lieferadresse, Produkt-, Mengen- und Preisangaben, Tourdaten sowie `ConfirmationStatus`.

Anschließend werden alle über die interne `Order`-ID zugeordneten `ConfirmationRequest`-Objekte geladen. Eine vorhandene `CustomerResponse` wird per `LEFT JOIN` zugeordnet, sodass auch unbeantwortete, ausstehende oder fehlgeschlagene Anfragen Bestandteil des Ergebnisses bleiben.

Die interne ID dient als technisches Ordnungsmerkmal, da kein gesonderter Erzeugungszeitpunkt gespeichert wird. Die Anfrage mit der höchsten ID wird als `currentRequest` ausgewiesen; die übrigen bilden in absteigender ID-Reihenfolge die `previousRequests`. `currentRequest` bedeutet dabei nicht, dass die Anfrage noch aktiv ist. Liegt keine Rückbestätigungsanfrage vor, bleibt dieser Bestandteil leer. Eine separate Paginierung der Historie ist im Prototyp nicht vorgesehen.

Der auftragsbezogene `ConfirmationStatus` ist vom Darstellungsstatus einer einzelnen Rückbestätigungsanfrage zu unterscheiden. Für diese werden die in Abschnitt 5.1.2 getrennt beschriebenen Merkmale `deliveryStatus`, `active` und eine gegebenenfalls vorhandene Kundenreaktion zu einem ansichtsspezifischen Status verdichtet.

Tabelle 3: Darstellungsstatus einzelner Rückbestätigungsanfragen

Zustand der Rückbestätigungsanfrage	Darstellungsstatus
<code>deliveryStatus = PENDING</code>	PENDING
<code>deliveryStatus = FAILED</code>	FAILED
<code>deliveryStatus = SENT, active = true, keine Kundenreaktion</code>	SENT
<code>deliveryStatus = SENT, responseType = CONFIRM</code>	CONFIRMED
<code>deliveryStatus = SENT, responseType = REJECT</code>	REJECTED
<code>deliveryStatus = SENT, active = false, keine Kundenreaktion</code>	NO_RESPONSE

Dadurch muss das Avisierungsdashboard die getrennten Zustandsinformationen nicht selbst kombinieren. Die Verdichtung bewahrt jedoch nicht immer die Ursache der Deaktivierung. Sowohl eine nach Fristablauf deaktivierte als auch eine durch eine neuere Avisierung abgelöste Anfrage wird ohne Kundenreaktion als `NO_RESPONSE` dargestellt, da keine separate Deaktivierungsursache gespeichert wird.

Die Historisierung ist auf anfragespezifische Daten begrenzt. Kommunikationskanal, Lieferfenster, Antwortfrist und technischer Versandstatus sowie gegebenenfalls Versand- und Ablaufzeitpunkte und Kundenreaktionen bleiben je `ConfirmationRequest` erhalten. Allgemeine Auftragsdaten werden dagegen aus dem aktuellen `Order` gelesen.

Die Detailansicht verbindet damit aktuelle Auftragsdaten mit einer Historie der Avisierungsversuche, stellt jedoch keine vollständige Versionierung des Auftrags dar. Frühere Kunden-, Adress-, Produkt- oder Tourdaten können nach einer Änderung nicht rekonstruiert werden.

5.6 Lieferverfolgung und Geocoding

5.6.1 Daten zur Lieferverfolgung und Fahrerposition

5.6.2 Zielkoordinaten und Geocoding

5.6.3 Bereitstellung für die Bestätigungsseite

5.7 Technische Konfiguration und lokale Integrationsumgebung

Die technische Konfiguration des Avisierungsservice ist auf drei Spring-Konfigurationsdateien verteilt. `application.yaml` enthält die gemeinsamen Einstellungen des Avisierungsservice, während `application-dev.yaml` die für die lokale Entwicklung benötigten Abweichungen ergänzt. Dazu gehören insbesondere die lokale Datenbankanbindung, zusätzliche Flyway-Ressourcen für Entwicklungsdaten sowie die Adressen der lokal angebundenen Systeme. `application-prod.yaml` enthält davon abweichende Einstellungen für den vorgesehenen Produktivbetrieb, beispielsweise für Datenbankzugang, Frontend-Adresse und Session-Cookie. Laufzeitabhängige beziehungsweise sensible Werte wie der Master-Key und die Twilio-Zugangsdaten werden nicht fest in der Anwendung hinterlegt, sondern über Umgebungsvariablen bereitgestellt. Für die lokale Entwicklung kann hierfür eine von der Versionsverwaltung ausgeschlossene `.env`-Datei verwendet werden.

Für die lokale Ausführung wurden das Webfrontend und der Avisierungsservice containerisiert und in die gemeinsame `docker-compose.yaml` aufgenommen. Der Container des Avisierungsservice wird mit dem Profil `dev` gestartet. Gemeinsam mit dem Webfrontend werden PostgreSQL, Mailpit, pgAdmin und der DISPO-Mock bereitgestellt. PostgreSQL stellt die Anwendungsdatenbank bereit, Mailpit nimmt lokal versendete E-Mails entgegen und der DISPO-Mock bildet die im Projekt benötigten DISPO-Schnittstellen für Callback und Lieferverfolgung nach. pgAdmin dient ergänzend zur manuellen Einsicht in die Datenbank, ist für den Betrieb des Avisierungsservice jedoch nicht erforderlich.

Die für den Containerbetrieb benötigten Adressen werden in Docker Compose an die interne Umgebung angepasst. Der Avisierungsservice verwendet dadurch für Datenbank, SMTP und DISPO die innerhalb des Compose-Netzwerks bereitgestellten Dienste anstelle der für einen direkten lokalen Start vorgesehenen `localhost`-Adressen. Für PostgreSQL wird vor dem Start des Avisierungsservice zusätzlich ein erfolgreicher Healthcheck abgewartet. Das `dev`-Profil ergänzt den regulären Flyway-Ablauf um ein entwicklungsbezogenes Callback zur Erzeugung lokaler Ausgangsdaten; die dafür benötigten SMTP- und DISPO-Adressen werden über konfigurierbare Platzhalter bereitgestellt. Die eigentliche Schemaentwicklung wurde bereits in Abschnitt 5.1.3 beschrieben.

Damit kann die für die Entwicklung der digitalen Lieferavisierung benötigte Umgebung gemeinsam über Docker Compose bereitgestellt werden. Externe Dienste wie Twilio und der Geocoding-Dienst bleiben außerhalb dieser lokalen Umgebung und werden bei Bedarf direkt

angebunden. Mailpit und der DISPO-Mock ersetzen damit nur die für die Entwicklung benötigten Schnittstellen, nicht die vollständigen Produktivsysteme.

6 Frontend – Konzeption und Implementierung

6.1 Einordnung des Frontends in das Gesamtsystem

6.2 Frontend-Architektur und technische Grundlagen

6.2.1 Technologieauswahl

6.2.2 Seiten-, Komponenten- und Routingstruktur

6.2.3 API-Anbindung und Zustandsverwaltung

6.2.4 Responsive Design und wiederverwendbare UI-Komponenten

6.3 Avisierungsdashboard

6.3.1 Geschützter Dashboard-Zugang

6.3.2 Tourübersicht und Statusdarstellung

6.3.3 Suche, Filterung und Paginierung

6.4 Auftragsdetailansicht und erneute Avisierung

6.4.1 Auftragsdaten und Avisierungshistorie

6.4.2 Erneute Avisierung, Validierung und Fehlerrückmeldung

6.5 Bestätigungsseite für Kunden

6.5.1 Tokenbasierter Zugriff und Lieferinformationen

6.5.2 Bestätigung, Ablehnung und Kundenkommentar

6.5.3 Abgeschlossene, abgelaufene und fehlerhafte Anfragen

6.6 Live-Tracking

6.6.1 Verfügbarkeit und Datenfluss

6.6.2 Karten- und Positionsdarstellung

6.6.3 Aktualisierung und Fehlerbehandlung

7 Test und Validierung

7.1 Teststrategie und Testumgebung

Die Validierung erfolgte schrittweise entlang der zentralen Anwendungsfälle. Zunächst wurden einzelne Komponenten isoliert überprüft, anschließend das Zusammenspiel von Frontend, Backend, Workflow Engine und Kommunikationskanälen getestet.

Ein besonderer Fokus lag dabei auf den Übergängen zwischen den Systemkomponenten, da dort fachliche Fehler oder inkonsistente Status besonders wahrscheinlich sind.

7.1.1 Testziele, Testebenen und Auswahl der Prüfschwerpunkte

7.1.2 Testwerkzeuge und reproduzierbare Testumgebung

7.2 Validierung des Avisierungsservice

7.2.1 Domänen- und Anwendungslogik

7.2.2 REST-Verträge, Eingabevalidierung und Fehlerantworten

7.2.3 Authentifizierung, Sitzungsverwaltung und Mandantenisolation

7.2.4 Persistenz- und Abfragemodell

7.2.5 Workflow-, Wiederholungs- und Nebenläufigkeitsszenarien

7.2.6 Ausgehende Schnittstellen und Rückkanäle

7.3 End-to-End-Tests

Die End-to-End-Tests prüfen die zentralen Benutzerabläufe der Kundenbestätigung, des Dispositions-Dashboards und der Lieferverfolgung über Frontend, Backend, E-Mail-Versand und DISPO-Rückmeldung hinweg. Sie werden mit Playwright im Chromium-Browser ausgeführt und befinden sich im Verzeichnis `frontend/tests`.

7.3.1 Einordnung der Szenariogruppen

Die Teststruktur orientiert sich an fachlichen Benutzerabläufen und nicht an einzelnen Frontend-Komponenten. Die Szenarien sind in die Bereiche Kundenbestätigung, Dispositions-Dashboard und Lieferverfolgung gegliedert.

Die Szenarien zur Kundenbestätigung umfassen den Versand und Inhalt der Bestätigungs-E-Mail, den Zugriff auf die Kundenseite, die Anzeige der Lieferdaten sowie die Bestätigung oder Ablehnung eines Liefertermins. Zusätzlich werden ungültige Bestätigungstoken, optionale Kundenkommentare und die Persistenz bereits abgegebener Antworten geprüft. Die Dashboard-Szenarien prüfen den geschützten Zugang, die Tourenübersicht, die Suche und Filterung, die Auftragsdetails, die Avisierungshistorie und die Auswahl des Kommunikationskanals für eine erneute Avisierung. Die Tracking-Szenarien prüfen die Verfügbarkeit und Aktualisierung der Lieferverfolgung.

7.3.2 Testumgebung und technische Struktur

Die Testsuite enthält insgesamt 29 Playwright-Tests. Davon bilden 28 Tests fachliche End-to-End-Szenarien ab. Ein zusätzlicher Setup-Test prüft vor der Ausführung die Erreichbarkeit der benötigten Dienste.

Die fachlichen Tests befinden sich in den Verzeichnissen `frontend/tests/confirmation`, `frontend/tests/dashboard` und `frontend/tests/tracking`.

Die Tests zur Kundenbestätigung sind auf folgende Dateien verteilt:

- `email.spec.ts` für den Versand und Inhalt der Bestätigungs-E-Mail

- `page.spec.ts` für die Anzeige der Kundenseite und das Verhalten bei ungültigen Bestätigungstoken
- `response.spec.ts` für Bestätigung, Ablehnung, Kundenkommentare, DISPO-Rückmeldungen und persistierte Abschlusszustände

Die Tests des Dispositions-Dashboards befinden sich in folgenden Dateien:

- `authentication.spec.ts` für den gültigen und ungültigen Dashboard-Zugang
- `overview.spec.ts` für die Tourenübersicht, Statusanzahlen, Suche, Filterung und Pagination,
- `order-detail.spec.ts` für Auftragsdetails, Anfragehistorie, Kundenreaktionen, Resend-Berechtigung und Kanalauswahl
- `settings.spec.ts` für das Laden und Testen der E-Mail- und SMTP-Einstellungen

Die Tests zur Lieferverfolgung befinden sich in `frontend/tests/tracking/tracking.spec.ts`. Sie prüfen die Tracking-Karte, die Fahrzeug- und Zielmarkierung, die Anzeige der Zieladresse, die Verfügbarkeit vor dem Liefertag und die Aktualisierung der Fahrerposition.

Wiederverwendbare technische Abläufe befinden sich unter `frontend/tests/helpers`. Dazu gehören das Erzeugen eindeutiger Testaufträge, das Auslesen von E-Mails aus Mailpit, das Öffnen einer authentifizierten Dashboard-Sitzung, die Prüfung von DISPO-Rückmeldungen und die Vorbereitung bestätigter Aufträge für Tracking-Szenarien.

Vor den fachlichen Tests führt Playwright den Setup-Test `setup/services.setup.ts` aus. Dieser prüft einmalig die Erreichbarkeit von Backend, Mailpit und DISPO-Mock. Die Prüfungen verwenden Polling, damit die Testausführung auf den Start der benötigten Docker-Dienste warten kann.

Dynamisch erzeugte Testaufträge erhalten mit `randomUUID()` eindeutige Auftragsnummern und E-Mail-Adressen. Die Dashboard-Tests verwenden vorbereitete DEMO-Seed-Daten. Playwright verändert dabei keine Datenbankzustände direkt.

7.3.3 Versand und Bereitstellung der Anfrage

Zur Prüfung des Versandablaufs wird über die geschützte DISPO-Schnittstelle eine Bestätigungsanfrage für einen eindeutigen Testauftrag erzeugt. Die dadurch ausgelöste Bestätigungs-E-Mail wird über den konfigurierten SMTP-Versand an Mailpit übermittelt. Dort wartet der Test auf den Eingang der Nachricht.

Anschließend wird überprüft, ob die E-Mail den Kundennamen, die Lieferadresse, das Produkt, die Bestellmenge, das Lieferzeitfenster, den Preis und den personalisierten Bestätigungslink enthält.

Damit bildet das Szenario den systemübergreifenden Ablauf von der Anlage eines Avisierungsauftrags bis zur Bereitstellung eines für den Kunden nutzbaren Bestätigungslinks ab. Bewertet wird dabei die tatsächlich in Mailpit eingegangene Nachricht.

7.3.4 Kundenzugriff und Anzeige der Lieferdaten

Nach dem Öffnen des Bestätigungslinks wird geprüft, ob die Bestätigungsseite alle für die Kundenentscheidung erforderlichen Informationen anzeigt. Dazu gehören Kundenname,

Lieferadresse, Produkt, Bestellmenge, Preis sowie Beginn und Ende des Lieferzeitfensters.

Zusätzlich wird geprüft, wie die Anwendung auf ein unbekanntes Bestätigungstoken reagiert. Sie zeigt in diesem Fall eine verständliche Fehlermeldung an und bietet keine Möglichkeit, den Liefertermin zu bestätigen oder abzulehnen.

7.3.5 Kundenreaktion und DISPO-Rückmeldung

Die Bestätigung und die Ablehnung eines Liefertermins werden als getrennte E2E-Szenarien geprüft.

Nach der Bestätigung muss die Anwendung eine sichtbare Erfolgsmeldung anzeigen. Zusätzlich wird geprüft, ob der DISPO-Mock eine Rückmeldung mit der externen Auftragsnummer und dem Status `CONFIRMED` erhält.

Ein weiterer Test prüft die Übermittlung einer optionalen Kundennachricht im Bestätigungsablauf. Der eingegebene Kommentar muss gemeinsam mit der `CONFIRMED`-Rückmeldung an den DISPO-Mock übertragen werden. Da für Bestätigung und Ablehnung dasselbe Kommentarfeld und derselbe Übertragungsweg verwendet werden, wird die Kommentarübermittlung nur einmal im Bestätigungsablauf geprüft.

Nach der Ablehnung muss die Anwendung ebenfalls eine sichtbare Rückmeldung anzeigen. Der DISPO-Mock muss dabei den Status `REJECTED` für die entsprechende externe Auftragsnummer erhalten.

Die Persistenz beider Abschlusszustände wird durch das erneute Öffnen des Bestätigungslinks geprüft. Der zuvor erreichte Status muss weiterhin angezeigt werden, während die Buttons zum Bestätigen und Ablehnen nicht mehr verfügbar sind. Im abgelehnten Zustand darf außerdem keine Tracking-Karte angezeigt werden.

7.3.6 Dashboard-Zugang

Für den Dashboard-Zugang fordert der Test mithilfe eines gültigen API-Schlüssels über die geschützte Backend-Schnittstelle einen Dashboard-Zugangslink an. Nach dem Öffnen dieses Links im Browser prüft der Test den Aufbau einer authentifizierten Dashboard-Sitzung, die Weiterleitung zum Dispositions-Dashboard und die Anzeige der Tourenübersicht.

Ein weiterer Test ruft die Anwendung mit einem ungültigen Zugangscode auf. In diesem Fall darf das Dashboard nicht angezeigt werden. Stattdessen muss die Anwendung eine verständliche Fehlermeldung ausgeben.

7.3.7 Tourenübersicht, Suche und Filterung

In der Tourenübersicht werden die verfügbaren Touren, die zugehörigen Aufträge und die aggregierten Statusanzahlen kontrolliert. Dabei werden bestätigte, abgelehnte und noch unbeantwortete Aufträge berücksichtigt.

Die globale Suche wird in getrennten Szenarien mit einem Kundennamen, einer Auftragsnummer und einer Lieferadresse ausgeführt. Dabei wird jeweils geprüft, ob der gesuchte Auftrag angezeigt und ein nicht passender Auftrag aus der Ergebnisliste entfernt wird.

Zusätzlich wird die kombinierte Filterung nach Tournummer und Bestätigungsstatus geprüft. Der Pagination-Test wechselt auf die zweite Ergebnisseite, kontrolliert den dafür

ausgeführten Backend-Aufruf und prüft die Anzeige der dort enthaltenen Ergebnisse.

7.3.8 Auftragsdetails und erneute Avisierung

Die Auftragsdetailseite zeigt die Stammdaten des Auftrags, den aktuellen Status und die aktuelle Bestätigungsanfrage. Die E2E-Tests prüfen unter anderem die Auftragsnummer, den Kundennamen, die E-Mail-Adresse, die Lieferadresse, das Fahrzeug, die Bestellmenge und den Kommunikationskanal.

Darüber hinaus wird die Avisierungshistorie geprüft. Historische Anfragen mit den Zuständen `REJECTED` und `NO_RESPONSE` sowie gespeicherte Kundenkommentare müssen auf der Detailseite sichtbar sein.

Die Verfügbarkeit der erneuten Avisierung hängt vom aktuellen Status ab. Für einen bereits bestätigten Auftrag muss der Button „Erneut senden“ deaktiviert sein. Für einen Auftrag mit dem Status `NO_RESPONSE` muss der Button dagegen aktiviert sein.

Für eine zulässige erneute Avisierung wird die Auswahl der Kommunikationskanäle E-Mail, SMS und WhatsApp geprüft. Nach der Auswahl eines Kanals muss die vorherige Auswahl aufgehoben und der aktive Kanal über das Attribut `aria-pressed` zugänglich gekennzeichnet werden. Der Button zum erneuten Senden muss dabei aktiviert bleiben.

Der Versand wird im E2E-Test nicht ausgelöst, da die Dashboard-Tests gemeinsam verwendete `DEMO`-Seed-Daten nutzen. Eine erneute Avisierung würde die aktuelle Anfrage und ihre Historie verändern und dadurch die Wiederholbarkeit der Tests beeinträchtigen.

Für SMS und WhatsApp wird ausschließlich die Auswahl in der Benutzeroberfläche geprüft. Ein tatsächlicher Versand über Twilio erfolgt nicht, um kostenpflichtige Nachrichten an externe Empfänger und eine Abhängigkeit von der Verfügbarkeit eines Drittdienstes zu vermeiden.

7.3.9 E-Mail- und SMTP-Einstellungen

Die E-Mail-Einstellungen des Dashboards werden in drei Szenarien geprüft.

Zunächst wird kontrolliert, ob die gespeicherten SMTP-Einstellungen vom Backend geladen und in der Benutzeroberfläche dargestellt werden.

Anschließend wird über den Button „Verbindung testen“ geprüft, ob das Backend die gespeicherte SMTP-Konfiguration erfolgreich verwenden kann. Nach erfolgreichem Abschluss muss eine entsprechende Meldung angezeigt werden.

Der dritte Test versendet eine SMTP-Test-E-Mail. Neben der sichtbaren Erfolgsmeldung wird geprüft, ob Mailpit tatsächlich eine neue Nachricht mit dem erwarteten Betreff erhalten hat.

7.3.10 Lieferverfolgung und Tracking-Verfügbarkeit

Für bestätigte Lieferungen am Liefertag wird geprüft, ob die Live-Tracking-Ansicht verfügbar ist. Die Tracking-Schnittstellen müssen gültige Ziel- und Fahrerkoordinaten liefern. In der Benutzeroberfläche müssen die Tracking-Karte sowie die Fahrzeug- und Zielmarkierung sichtbar sein.

Ein weiteres Szenario prüft, ob die Lieferadresse als Zieladresse angezeigt wird und die Zielmarkierung auf der Karte sichtbar ist.

Bei einer bestätigten Lieferung mit einem zukünftigen Lieferdatum meldet die Tracking-Schnittstelle, dass die Lieferverfolgung noch nicht verfügbar ist, und liefert keine Zielkoordinaten. Die Anwendung zeigt daher weder die Tracking-Karte noch die Fahrzeug- und Zielmarkierung an. Stattdessen wird der Kunde darauf hingewiesen, dass die Tracking-Informationen erst am Liefertag verfügbar sind.

Abschließend wird die Aktualisierung der Fahrerposition über den Button „Aktualisieren“ geprüft. Dabei müssen neue Fahrerkoordinaten geladen und der angezeigte Lieferstatus beziehungsweise die verbleibende Entfernung aktualisiert werden. Die Karte sowie die Fahrzeug- und Zielmarkierung müssen weiterhin sichtbar bleiben.

8 Projektmanagement und Projektreflexion

8.1 Projektmanagement

8.1.1 Projektphasen

Das Projekt wurde in klar abgegrenzte Phasen unterteilt, um Analyse, Konzeption, Umsetzung und Validierung strukturiert durchführen zu können. Die Übergänge zwischen den Phasen waren teilweise iterativ, da Erkenntnisse aus der Implementierung wieder in die Ausgestaltung des SOLL-Prozesses eingeflossen sind.

Phase	Zeitraum	Status
Analyse und Anforderungsabstimmung	xx.xx - xx.xx	abgeschlossen
Architektur und Prozessmodellierung	xx.xx - xx.xx	abgeschlossen
Implementierung	xx.xx - xx.xx	abgeschlossen
Integration und Test	xx.xx - xx.xx	abgeschlossen
Abschluss und Ergebnisaufbereitung	xx.xx - xx.xx	abgeschlossen

8.1.2 Projektplan

Der Projektplan diene als gemeinsame Orientierung für inhaltliche Meilensteine, Abstimmungstermine und die Reihenfolge der Umsetzung. Besonders hilfreich war die frühe Sichtbarkeit von Abhängigkeiten zwischen Prozessmodell, Schnittstellen und Benutzeroberfläche.

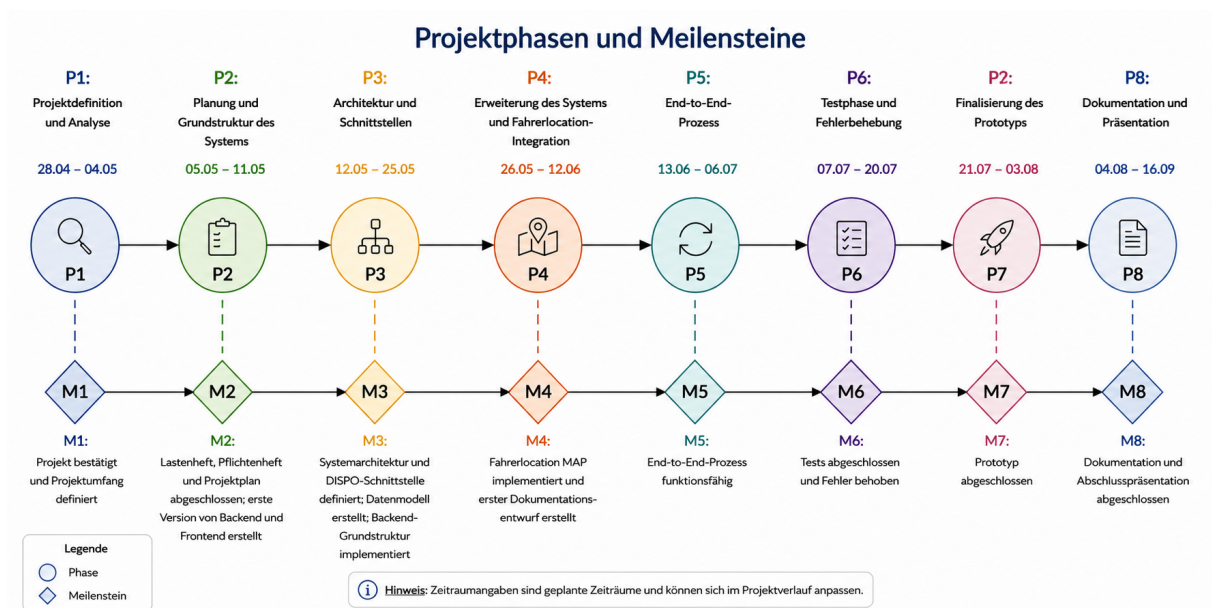


Abbildung 9: Projektplan

8.1.3 Projektaufwand

Der dokumentierte Aufwand verteilt sich auf Analyse, Konzeption, Implementierung, Test und Abstimmung mit dem Auftraggeber. Die Tabelle dient als Beispielstruktur zur Erfassung der geleisteten Stunden und kann mit den tatsächlichen Projektdaten vervollständigt werden.

Datum	Person	Tätigkeit	Stunden
xx.xx.2026	Name 1	Anforderungsanalyse und Recherche	4
xx.xx.2026	Name 2	Backend-Implementierung	6
xx.xx.2026	Name 3	BPMN-Modellierung und Camunda-Integration	5
xx.xx.2026	Name 1	Frontend-Umsetzung und UI-Anpassungen	5
xx.xx.2026	Team	Integrationstest und Ergebnisabstimmung	4

8.2 Projektreflexion

8.2.1 Herausforderungen

Herausfordernd war insbesondere die Abstimmung an den Schnittstellen zwischen fachlicher Prozessbeschreibung und technischer Umsetzung. Bereits kleine Unklarheiten bei Statusübergängen oder Rückmeldewegen hatten direkte Auswirkungen auf mehrere Systemkomponenten.

Auch die Konsistenz zwischen Benutzerführung, Datenmodell und Workflowlogik erforderte wiederholte Abstimmungen.

8.2.2 Lessons Learned

Für vergleichbare Projekte empfiehlt es sich, Statusmodelle, Ereignisse und Verantwortlichkeiten frühzeitig gemeinsam zu definieren und schriftlich festzuhalten. Darüber hinaus hat sich gezeigt, dass ein visuelles Prozessmodell in Kombination mit einer frühen technischen Teilimplementierung sehr hilfreich ist, um Missverständnisse schnell sichtbar zu machen.

9 Projektresultate

9.1 Gesamtergebnis

Die Projektarbeit führte zu einem konsistenten Prototypen, der die wesentlichen Anforderungen des definierten SOLL-Prozesses abbildet.

- Funktionsfähiger Prototyp
- Frontend-Backend-Kommunikation
- Automatisierte Kommunikation über verschiedene Kanäle
- Digitale Rückbestätigung von Lieferterminen
- Statusverwaltung
- Workflow-Steuerung
- Avisierungsdashboard für die Disposition
- Bestätigungsseite für Kunden
- Liefervorgang bzw. Fahrerkartenansicht

9.2 Grenzen des Projektergebnisses

9.3 Fazit und Ausblick

Das Projekt konnte sowohl aus technischer als auch aus organisatorischer Sicht erfolgreich abgeschlossen werden. Neben dem entwickelten Prototypen entstand eine belastbare Grundlage für die weitere fachliche und technische Ausarbeitung des Systems.

Die Kombination aus Prozessmodellierung, Webanwendung und automatisierter Kommunikation erwies sich als geeigneter Ansatz für die Digitalisierung des betrachteten Anwendungsfalls.

10 Nutzung von Künstlicher Intelligenz

11 Verwendete Hilfsmittel